

Certified Instruction Selection at Scale

Validating LLVM against the Authoritative RISC-V Semantics using Certified BitBlasting

Sarah Kuhn (ETH Zurich BSc. thesis, long paper format)

Effective and correct instruction selection is crucial for converting high-level languages to efficient and trustworthy machine code. Widely adopted programming languages such as C++, Rust, Lean, and even Python trust LLVM’s backend to reliably translate a generic intermediate representation into high-performance assembly code. The resulting performance expectations push instruction selectors to incorporate a wide range of sophisticated, optimizing rewrites, which remain a persistent source of bugs, despite careful testing. Certifying the entire compiler backend correct, e.g., in CompCert, prevents such miscompilation at the cost of a high-manual proof burden and a niche end-product. Alternatively, recent work has pursued SMT-based verification in a widely used LLVM backend by recycling verification tools built for the compiler’s middleend. While automated, they rely on unverified tooling and on manually derived ISA semantics, shifting trust to unverified SMT solvers and subjective ISA interpretations, thereby weakening correctness guarantees. We combine maximal trust with scalable, practical verification and present certified instruction selection for a subset of LLVM IR, automatically verified against authoritative ISA semantics. We integrate and certify LLVM backend rewrites, support extensible pattern integration, and a minimal verification effort, fully preventing unsound instruction selection rules. By leveraging the end-to-end verified bit-blaster `bv_decide`, we avoid unverified verification tooling and only trust the minimal kernel of an ITP. Our prototype shows that instruction selection, automatically certified against official ISA semantics, can deliver trustworthy software to compiler backends while supporting rewrites and the development speed of mainstream compilers.

1 Introduction

Modern compilers apply a suite of performance-oriented, iterative optimizations in their middleend, ranging from localized code rewrites (Figure 1) to interprocedural analyses spanning entire programs. Similar optimizations occur in the backend, however tightly interleaved with code generation passes, such as instruction selection, that map the compiler *Intermediate Representation* (IR) onto the target *Instruction Set Architecture* (ISA). This mapping is expected to preserve the semantics of the original program, but typically lacks formal correctness guarantees, remaining a persistent source of bugs (Figure 2). As a result, semantic divergences between compiled code and its source remain a recurring pathology within compiler frameworks.

<pre>1 define i32 @unopt(i32 %a) { 2 %0 = sub i32 %a, 0 3 ret i32 %0 4 }</pre>	<pre>1 define i32 @opt(i32 %a) { 2 3 ret i32 %a 4 }</pre>
(a) Original function	(b) Optimized function

Fig. 1. Compilers replace a local code pattern, e.g., a subtraction by zero (a), with more efficient code that drops the unnecessary subtraction (b), which is referred to as a peephole optimization. Instruction selectors in modern compilers contain many such peephole optimizations, which are typically not formally verified.

Semantic loss during compilation stems from logical implementation mistakes or a lack of formalization (Figure 3) regarding what constitutes correct language behavior, e.g., in the case of overflow, thereby fostering inconsistent compilation by giving developers the freedom to ambiguously define the effects of their optimizations.

One proposed solution is to develop fully verified compilers, e.g., CompCert [21], a fully verified compiler for C, and establish maximal trustworthy compilation pipelines. However, the high,

50	1	<code>smul.with.overflow(%x,1)</code>	1	<code>smul.with.overflow(%x,1):= false</code>
51	2	<code>\\indicates whether x * 1 results in overflow.</code>	2	<code>\\incorrectly optimized to always return false</code>
52				
53	(a) Unoptimized function		(b) Wrongly optimized code	

Fig. 2. Incorrectly optimizing the overflow flag for signed multiplication by 1. The optimization holds except for 1-bit integers. For 1-bit integers, the signed interpretation of 1 is (-1). In this case, multiplying (-1) by (-1) yields 1, which triggers an overflow, as 1 is not part of {-1, 0}.

```

58 1 def execute_SHIFTIOP (shamt : BitVec 6) (op : sop) (rs1_val : BitVec 64) : BitVec 64 := match op with
59 2   | sop.RISCV_SLLI => (shift_bits_left rs1_val shamt)
60 3   | sop.RISCV_SRLI => (shift_bits_right rs1_val shamt)
61 4   | sop.RISCV_SRAI => (shift_bits_right_arith (rs1_val) shamt)

```

Fig. 3. Formalization of the RISC-V 64-bit shift-immediate instructions. Each instruction is mapped to its corresponding bit-level operation, providing a precise, unambiguous definition.

blocking proof burden makes them hard to extend, slow to adapt, and relegated to niche tools far from general-purpose compilation. This is contrasted by modern production compilers, which advance fast-paced through continuous development by their community, e.g., LLVM [22] used to compile C, Rust, and Lean. A drawback is that the high-speed, collaborative, open-source nature favors subtle bugs, and a bug in a language-agnostic compiler toolkit like LLVM compromises the correctness of programs across any language that relies on it. One such example is the Lean programming language and proof assistant [9], whose compiler is part of the trusted code base, and compilation results are used to reason in proofs. Consequently, soundness directly hinges on the trustworthiness of the LLVM-based compiler. Similarly, Rust relies on the LLVM toolchain; once compiled, preservation of properties implemented at the Rust language level relies on the correctness of LLVM [36].

Previous verification has targeted the LLVM ecosystem, however, focusing on the higher stack around the first-class LLVM IR. Little effort has focused on backend certification, despite the backend being considered a “highly optimizing compiler in its own” [3]. Any correctness guarantees established in the middleend can be invalidated by bugs in the backend. Regehr et al. [3] investigated the 64-bit ARM AArch backend and uncovered a multifold of bugs, highlighting that these low-level components remain as the most error-prone part of the compilation pipeline. However, while automatic, their approach relied on SMT solvers implemented by large, unverified codebases with a history of bugs and a reuse of IR verification tooling to certify lifted assembly. Nuno et al. [23] noted that 20% of issues encountered while detecting LLVM middleend misoptimizations were due to SMT solver bugs. Moreover, backend verification is commonly based on handwritten ISA semantics, e.g. in CompCert, requiring trust in the author’s interpretation of the ISA and thereby performing verification on fragile foundations [3, 20]. Instead, we propose to build trustworthy instruction selection automatically verified against authoritative ISA semantics using reliable, certified verification tooling. This prevents shifting trust from the compiler to other unverified codebases, proves full correctness as opposed to validating single executions, and takes a step towards dependable backends as the foundation for the modern software stack.

Our Approach: We present a prototype of an extensible, verified instruction selection pipeline for a subset of LLVM IR targeting the 64-bit RISC-V processor based on LLVM rewrites, authoritative ISA semantics and certified verification tools. The system is implemented within the Lean *Interactive Theorem Prover* (ITP) and lowers the pure arithmetic fragment of LLVM IR into an authoritative RISC-V assembly dialect while preserving semantics in every case. We model a multi-level IR and

ISA in Lean, and investigate the use case of mechanized processor models for compiler verification. We extract and verify LLVM’s instruction selection patterns as well as optimizing rewrites, to reimplement a subset in our pipeline. Our design emphasizes modularity, extensibility, and a low proof burden, which we showcase by example in the next section (Section 2). We avoid unverified SMT solvers for verification, instead rely on an ITP with a minimal trusted kernel and a verified bit-blaster to discharge our proof obligations. Finally, we provide a complete compilation flow invokable over command line, that integrates our verified instruction selection as a backend pass.

Our contributions are:

- `riscv`, a **Single Static Assignment (SSA)**-based ISA dialect with **authoritative, bit-blastable RISC-V semantics**, serving as a compilation target that allows to verify lowerings to the RISC-V in Lean (Section 4)
- A **library of certified instruction selection patterns** for integer arithmetic extracted from LLVM with **proof automation** that certifies them against official RISC-V semantics using `riscv` (Section 5)
- **Verification of optimizing GlobalISel rewrites** to enhance the certified pattern library with LLVM optimizations (Section 7)
- A **verified instruction selector from LLVM IR to RISC-V** derived from our library of certified lowering patterns integrated into an unverified backend (Section 8)
- An evaluation showing **86% coverage** of the RISC-V dialect with **69% bit-blastable semantics**, demonstrating how our infrastructure **enhances trust in LLVM** and whether past LLVM defects would have been flagged. We show that our verified selector is extensible and produces **code sizes comparable to LLVM’s modern instruction selector GlobalISel** (Section 10)

2 The Idea: Certified Instruction Selection

We propose a certified instruction selector for a fragment of LLVM IR that combines minimal proof burden with authoritative ISA semantics and automated verification (Figure 4). Mirroring LLVM, the selector is organized around a collection of rewrite patterns, each specifying a match-and-replace pattern capturing a LLVM IR to RISC-V lowering. To ensure a verified yet extensible instruction selector, we allow the addition of new certified rewrites by providing proof automation that discharges the verification burden whenever possible. Aiming for maximal trust, we verify each **Instruction Selection (ISel)** pattern against authoritative RISC-V ISA semantics as implemented in the official, mechanized Sail RISC-V ISA specification. Moreover, we propose to rewrite such into bitvector semantics compatible with highly optimized solvers for bitvector reasoning, such as `bv_decide`, while provably retaining authoritative status. By minimizing the proof burden and supporting MLIR syntax within the Lean ITP through Lean-MLIR, patterns can be formalized using syntax familiar from MLIR compiler development. The individually verified rewrites are composed into a collection forming the basis of a certified instruction selector, built atop the end-to-end verified optimizer in Lean-MLIR. This yields a verified instruction selection pass that we integrate as the trustworthy component into an otherwise unverified backend, enabling a complete compilation flow for LLVM IR to RISC-V textual assembly with an extensible, certified ISel core.

3 Background

4 Compiler Abstractions Within An ITP

A compiler systematically leverages abstraction such as **IRs** when translating high-level programs into low-level machine code. For example, an **IR** such as LLVM IR abstracts over several source languages, facilitating optimization and transformation. Machine code abstracts hardware-specific

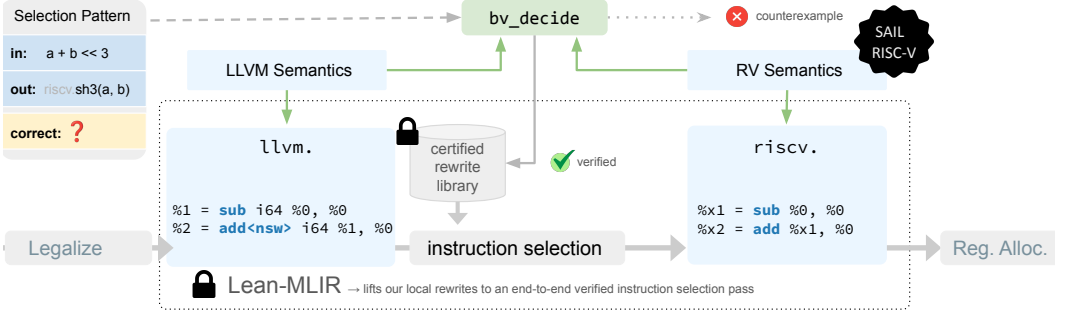


Fig. 4. Verified Instruction Selection for LLVM IR to RISC-V machine code. The selection patterns and optimizing rewrites are automatically verified against authoritative RISC-V ISA semantics using the verified `bv_decide` bit-blaster. The backend pass is implemented in Lean using Lean-MLIR and can be extended with new rewrites requiring little proof burden, making the infrastructure extensible and maximally trustworthy.

details through *ISAs*. Situated at the core of a compiler backend, instruction selection serves as the bridge between such abstraction layers, mapping *IR* operations to machine instructions.

We describe the Lean formalization of the three *IRs* used in our instruction selection, allowing us to reason about the correctness of an LLVM IR to RISC-V lowering. We first explain the idea of each *IR* followed by its implementation and conclude by stating our correctness claim, which specifies when a compiler lowering is correct.

4.0.1 Overview Of The IRs.

- `riscv`: `riscv` is the preregister-allocated SSA-based *ISA IR* that abstracts RISC-V assembly and supports authoritative, bit-blastable semantics extracted from the Sail RISC-V processor model for each of its operations.
- `llvm`: The `llvm` dialect formalizes the arithmetic fragment of LLVM IR, including its poison semantics, enabling machine-checked reasoning about a realistic *IR* in Lean. The `llvm` dialect is not designed within this thesis, but a preexisting implementation described in the background (??) and in [4]. We build on it to represent LLVM IR within our instruction selector.
- `llvmAndriscv`: `llvmAndriscv` is a hybrid *IR* containing an abstraction for LLVM IR and the RISC-V instruction set, provided by `llvm` and `riscv`. `llvmAndriscv` leverages MLIR’s concept of progressive lowerings and repurposes it for reasoning about and implementing instruction selection.

4.1 SSA-Based IR with Verified Semantics For The RISC-V ISA

`riscv` is an SSA-based assembly *IR* in Lean that models a comprehensive subset of the RISC-V 64-bit *ISA*. In contrast to low-level compiler *IRs*, such as the RISC-V dialect in xDSL [12] or the x86vector dialect in MLIR [28], which leave the semantics of operations to be defined by compiler mappings, `riscv` follows a declarative approach. It integrates authoritative ISA semantics directly into the dialect design and provides a mechanization of existing RISC-V dialects, e.g., `riscv` in xDSL [25]. In `riscv`, each operation is assigned to *ISA* instruction semantics extracted from the Sail RISC-V processor mechanization and thereby `riscv` enables the use of an official formal processor model, commonly used in processor verification, to verify backend compilation. `riscv` operations can be reasoned about in terms of their integrated semantics (Figure 5), which are adapted such that

the resulting proof goals can be automatically discharged by the verified bit-blaster, `bv_decide`, in Lean.

```

1  %0 = bext %r1, %r2 : !riscv.reg
2
3  denoted as:
4  BitVec.setWidth 64
5  (match
6    BitVec.and rs1_val ((BitVec.setWidth 64 1#1)
7      <<< (BitVec.extractLsb 5 0 rs2_val)) != 0#64 with
8    | true => 1#1
9    | false => 0#1)

```

Fig. 5. The RISC-V assembly operation `bext` (single-bit extract) modeled in the SSA-based `riscv` dialect with authoritative semantics accessible in Lean, allowing to reason about the instruction.

4.1.1 Representing The ISA. As its name suggests, `riscv` models a 64-bit **ISA** and therefore operates over 64-bit bitvectors representing the contents of 64-bit general-purpose registers. The operands and result are determined by the instruction type, for example, R-type instructions such as `add` operate on two 64-bit operands, whereas I-type instructions such as `lui` combine a 64-bit register value with a fixed-width immediate. immediates are modeled as bitvectors whose width is determined by the **ISA** specification. Besides instruction type, ISA operations are categorized into extensions. `riscv` models the RV64I base **ISA** for a 64-bit RISC-V processor, extended with the standard M extension and a subset of the B extension (Figure 6). These extensions capture integer arithmetic and effective bit-manipulation primitives most frequently exploited by compilers in practice (Figure 6)¹ and are all native hardware instructions directly implemented by RISC-V processors. Generally, `riscv` can support operations where inputs and outputs can be modeled as operand values and therefore currently can’t implement the RISC-V load, store, and fence instructions.

ISA Subset	Function	# Instructions	riscv
RV64I	Base ISA	33	91%
M	Mul/Div	13	100%
B	Bitmanipulation	41	82%
Total		87	88%

Fig. 6. Overview of the supported 64-bit ISA fragment by `riscv`.

Crucially, `riscv` operations act on register values, not registers, and operands denote the content of a register, e.g., each operation returns a 64-bit bitvector, which corresponds to the value to be written into the destination register. This design makes `riscv` a pure dialect focused on execution semantics, independent of the origin of the values. Additionally, this design allows for **SSA**, providing a thin abstraction layer between first-class compiler **IRs**, commonly in **SSA**, and `riscv`. **SSA** allows to separate concerns with an eye towards simpler verification: compiler transformations over the dialect, like instruction selection, can focus purely on instruction mapping, which is unaffected

¹According to the SiFive chief architect, the M extension together with ZBA, ZBB, and ZBC (subsets of the B extension) are the most widely used and most relevant RISC-V extensions for general-purpose compilation.

by whether the operands are virtual or physical registers. This allows us first to reason about instruction mapping independently and then separately reason about register allocation, enabling a modular approach to verification.

RISC-V assembly sequences can be directly written into the Lean **ITP** frontend via `riscv`, using its implemented MLIR syntax support, which is familiar from MLIR compiler development (Figure 7). Parsing `riscv` as MLIR syntax allows its textual dialect representation to be integrated into any tool that uses MLIR syntax, e.g. *mlir-opt* [29] or xDSL’s pipeline [12]. Besides supporting generic MLIR syntax, we implemented an optional custom syntax close to textual RISC-V assembly (Figure 8). `riscv` can therefore be written directly in Lean using two established syntax forms.

```
1  ^bb0(%r1 : !riscv.reg, %r2 : !riscv.reg):
2  %0 = "and" (%r1, %r2) : (!riscv.reg, !riscv.reg) -> !riscv.reg
3  %1 = "div" (%0, %r1) : (!riscv.reg, !riscv.reg) -> !riscv.reg
4  %2 = "addi" (%1)(imm = 10 : !i12) : (!riscv.reg) -> !riscv.reg
5  "ret" (%2) : (!riscv.reg) -> ()
```

Fig. 7. RISC-V dialect representation support in Lean using MLIR generic syntax for IRs. `and` and `div` are R-type instructions, while `addi` is of I-type relying on an immediate encoded as an instruction attribute.

```
1  ^bb0(%r1 : !riscv.reg, %r2 : !riscv.reg):
2  %0 = and %r1, %r2 : !riscv.reg
3  %1 = div %0, %r1 : !riscv.reg
4  %2 = addi %1, 10 : !riscv.reg
5  ret %2 : !riscv.reg
```

Fig. 8. Custom syntax for the RISC-V dialect in Lean mirroring traditional textual assembly using SSA.

Figure 10 exemplifies the `riscv` model of the I-type assembly instruction sequence encoded in Figure 9 where a left shift with an immediate operand (`slli`) is performed. The operation takes a 64-bit register value (`!riscv.reg`) as input and an immediate, producing a 64-bit register value as a result. Notice that the example contains a shift by 65, which exceeds the width of a 64-bit

```
1  slli x2, x1, 65
```

Fig. 9. Textual RISC-V assembly such as LLVM’s *llc* compiler would output. `x2` is the source register and `x1` the destination register, while 65 specifies the shift amount.

```
1  ^bb0(%x1 : !riscv.reg):
2  %0 = slli %x1, 65 : !riscv.reg // directly embedded semantics in Lean.
3  ret %0 : !riscv.reg
```

Fig. 10. Representation of the RISC-V assembly `slli` instruction in `riscv` with underlying access to the Sail execution semantics within Lean. `riscv` is in SSA, therefore the return value is captured in the SSA value `%0` while `%x1` specifies the value within the source register.

RISC-V machine register. The semantics of such an instance of `slli` are not obvious, since different ISAs specify the behavior of an overshift differently, e.g., by wrapping the shift amount or by hard-coding the result to a constant. In `riscv`, the behavior is coupled to the operation, not fallible to people misreading the specification, through the direct embedding of the RISC-V ISA instruction

semantics. Additionally, the semantic denotation of each operation is explicitly extractable in Lean, making it accessible for mechanized proofs about assembly programs. The next section discusses the semantics within `riscv` in more detail.

4.1.2 Embedding Authoritative ISA Semantics. Coupling `ISA` semantics into the mechanization of dialect operations makes the meaning of a program explicit and machine-checkable, e.g., enabling compiler transformations over the dialect to be verified against these embedded semantics when compiling onto the dialect. In `riscv`, after defining the instruction type and specifying instruction operands (Section 4.1.4), each `ISA` operation is assigned a function that defines its behavior with respect to inputs and operation attributes (Figure 11).

```

1  \ mapping each operation onto a mathematical function.
2  slli shamt, regs => SHIFTIOP_RISCV_SLLI_semantics shamt (regs.getN 0)
3  add, regs => RTYPE_RISCV_ADD_semantics (regs.getN 0) (regs.getN 1)
4
5
6  \ the underlying definition over bitvectors.
7  (shamt : BitVec 6) (rs1 : BitVec 64) => rs1 <<< shamt (for slli)
8  (rs1 : BitVec 64) (rs2 : BitVec 64) => rs1 + rs2 (for add)

```

Fig. 11. The idea of mapping each `ISA` instruction onto a mathematical function that specifies its execution semantics using the operation input (`regs`) and attributes (`shamt` in `slli`). In `riscv`, these functions are extracted from `Sail RISC-V`.

Obtaining Authoritative And Practical RISC-V Semantics In Lean. Hence, instead of defining `ISA` semantics by hand, as in previous verification projects, e.g. [3], we use the `Sail RISC-V` processor model adapted as the `ISA` specification by RISC-V official to obtain trustworthy semantics. However, instead of directly using the semantics in the form specified in the `Sail` model, which are difficult to work with, e.g., due to the large code size and privileged processor constructs, reasons why compiler verification projects fallback onto manually-driven `ISA` models, we extract the `Sail` semantics and then define a lightweight, bit-blastable subset of RISC-V `ISA` semantics in Lean with proven correspondence to the official `Sail` specification. The extracted subset of `ISA` operation semantics currently focuses on `riscv` operations (base `ISA`, B and M), however, the approach is applicable to other parts of the RISC-V `ISA`, e.g., floating point.

Our procedure to provably extract the corresponding RISC-V `ISA` definition and assign it to dialect operations is described below. We provide a high-level description of each step with a concrete implementation example using Lean-specific syntax detailed in [31].

- *Locate semantics.* Navigate the Lean translation of the `Sail RISC-V` model to identify the semantic execution functions corresponding to the selected `ISA` operations (Figure 12).
- *Isolate pure functions.* Execution functions are rewritten into pure forms by replacing register reads with explicit input parameters and expressing register writes as returned values (Figure 13). This yields a function that specifies an instruction solely in terms of its effect on input values, independent of the underlying registers. The pure execution function can then be invoked from a skeleton (Figure 14) that manages register access, extracts the relevant values, and supplies them as inputs. In this way, the computation embodied within an instruction semantics is separated from register read/write operations that supply the value.
- *Prove equivalence.* Construct machine-checked proofs in Lean establishing that the rewritten pure semantics are equivalent to the original `Sail`-based execution semantics (Figure 15).

```

344 1 def execute_MUL (rs2 : regidx) (rs1 : regidx) (rd : regidx) (mul_op : mul_op) : SailM Retired := do
345 2   let rs1_val ← do (rX_bits rs1)
346 3   let rs2_val ← do (rX_bits rs2)
347 4   let rs1_int : Int :=
348 5     if mul_op.signed_rs1
349 6     then (BitVec.toInt rs1_val)
350 7     else (BitVec.toNat rs1_val)
351 8   let rs2_int : Int :=
352 9     if mul_op.signed_rs2
353 10    then (BitVec.toInt rs2_val)
354 11    else (BitVec.toNat rs2_val)
355 12   let result_wide := (to_bits (2 *i xlen) (rs1_int *i rs2_int))
356 13   let result :=
357 14     if mul_op.high
358 15     then (Sail.BitVec.extractLsb result_wide ((2 *i xlen) -i 1) xlen)
359 16     else (Sail.BitVec.extractLsb result_wide (xlen -i 1) 0)
360 17   (wX_bits rd result)
361 18   (pure RETIRE_SUCCESS)

```

Fig. 12. Execution semantics of the multiplication (mul) ISA operation in the Sail–Lean RISC-V processor model. The function takes two register indices (regidx) and a parameter specifying the multiplication variant (e.g., signed, unsigned, high/low bits). The final line, *pure RETIRE_SUCCESS*, indicates that the operation is encoded in the Sail monad as a pure instruction with no side effects beyond updating the destination register. This example illustrates both the style of semantic definitions in the Sail RISC-V model and the complexity and overhead they introduce.

```

364 1 def execute_MULW_pure64 (rs2_val : (BitVec 64)) (rs1_val : (BitVec 64)) : BitVec 64 :=
365 2   let rs1_val32 := (Sail.BitVec.extractLsb rs1_val 31 0)
366 3   let rs2_val32 := (Sail.BitVec.extractLsb rs2_val 31 0)
367 4   let rs1_int : Int := (BitVec.toInt rs1_val32)
368 5   let rs2_int : Int := (BitVec.toInt rs2_val32)
369 6   let result32 := (Sail.BitVec.extractLsb (to_bits 64 (rs1_int *i rs2_int)) 31 0)
370 7   let result : xlenbits := (sign_extend (m := ((2 *i 3) *i 8)) result32)
371 8   result

```

Fig. 13. Replacing each register with the register value, and instead of writing back to a register, we return the value to be written.

```

373 1 def skeleton_binary (rs2 : regidx) (rs1 : regidx) (rd : regidx) (execute_func : BitVec 64 -> BitVec 64 -> BitVec 64) : SailM
374 2   Retired := do
375 3   let rs1_val ← rX_bits rs1
376 4   let rs2_val ← rX_bits rs2
377 5   let result := execute_func rs1_val rs2_val
378 6   wX_bits rd result
379 7   pure RETIRE_SUCCESS

```

Fig. 14. The skeleton used to separate register reads and writes from the actual computation embodied by the ISA instruction.

```

382 1 theorem mul_eq (rs2 : regidx) (rs1 : regidx) (rd : regidx) (mul_op : mul_op) :
383 2   Functions.execute_MUL (rs2) (rs1) (rd) (mul_op)
384 3   = skeleton_binary rs2 rs1 rd (fun val1 val2 => execute_MUL_pure64 mul_op val2 val1)
385 4   := by
386 5   unfold Functions.execute_MUL skeleton_binary execute_MUL_pure64
387 6   rfl

```

Fig. 15. Proving equivalence between the definition in Sail (*Functions.execute_MUL*) and the extracted *execute_MUL* function that captures the value semantics of the ISA operation while the skeleton performs the register read/write.

- *Extraction of semantics in terms of integers/naturals and bitvectors.* We first extract the semantics expressed over integers and naturals (Figure 16) and then rewrite it into equivalent fixed-width bitvector formulas (Figure 17), each supported by a Lean proof. When a single Sail function covers multiple instruction variants (e.g., signed vs. unsigned multiplication), this step is performed separately for each variant to obtain semantics that can be assigned to the corresponding individual instruction.

```

1 theorem execute_MUL_pure64_ftt (rs2_val : BitVec 64) (rs1_val : BitVec 64) :
2   execute_MUL_pure64 (mul_op := { high := False, signed_rs1 := True, signed_rs2 := True }) rs2_val rs1_val
3   = BitVec.extractLsb' 0 128 (BitVec.extractLsb' 0 128 (BitVec.ofInt 129 (rs1_val.toInt * rs2_val.toInt)))
4   := by
5   unfold execute_MUL_pure64
6   simp only [decide_false, Bool.false_eq_true, reduceIte, decide_true, Sail.BitVec.extractLsb,
7   Functions.xlen, to_bits, sail_hPow_eq, Int.reduceToNat, Int.reducePow, Int.reduceMul,
8   Int.reduceSub, get_slice_int]

```

Fig. 16. Extraction of the integer/natural-number statement capturing the semantics of the RISC-V mul instruction. The extracted statement is proven equivalent to the semantics defined by the pure function in Figure 13.

```

1 def pure64_MUL_ftt (rs2_val : BitVec 64) (rs1_val : BitVec 64) : BitVec 64 :=
2   rs2_val * rs1_val

```

Fig. 17. The final extracted semantics of the RISC-V mul instruction are defined purely over the bitvector domain and are visibly simpler than the original Sail definition. Moreover, expressing the semantics solely over fixed-width bitvectors enables the use of verification tooling optimized for bitvector reasoning, as the domain is bounded.

This procedure enabled us to transform the large, monadic execution functions from the Sail RISC-V model into simplified, pure functions over register values for each ISA instruction supported in riscv. We avoid dependence on manually formalized ISA semantics, which typically lack formal correctness guarantees, and instead extract the relevant fragment of a formal RISC-V processor model, thereby omitting non-userspace features, reducing the overhead of full system models, and limiting the technical burden to what is necessary for capturing instruction-level semantics.

Next, we integrated the extracted functions into the dialect definition of riscv and map each instruction onto its semantic function, e.g., multiplication (mul) onto pure64_MUL_ftt (Figure 17). Finally, we obtain a software ISA dialect that serves as an intermediate compilation target, from which we can extract a precise mathematical term specifying the semantics of the lowered program.

4.1.3 Pseudo-instructions. Besides native ISA instructions, riscv supports pseudo-instructions, which are assembler-level aliases that improve readability of textual assembly, but are not executed directly by the processor. Instead, compiler backends such as LLVM emit them in assembly output, after which the assembler expands them into concrete base instructions during binary generation. For example, the load immediate (li) pseudo-instruction abstracts the process of loading a constant, which requires a sequence of base instructions such as lui, addi, and additional arithmetic operations depending on the constant’s size. Thus, a single pseudo-instruction can correspond to an entire sequence of native operations. Despite their presence in the output of a compiler like LLVM or GCC, pseudo-instructions are only specified in the RISC-V assembly manual or implicitly through the compiler implementation, absent from the official Sail RISC-V processor model. To support them in riscv, we manually derived their semantics from the assembly manual and assigned them

using aliasing to authoritative semantics of native instructions. For instance, `not` is modeled as a specific instance of `xor`, which is a native ISA instruction (Table 1). In total, `riscv` supports 11 pseudo-instructions that alias base ISA, M, or B extension instructions, extending `riscv` to include instructions exclusively targeted by compilers.

Pseudo-instruction	Base Instruction	Semantics Based On Native Instructions
<code>not rd, rs</code>	<code>xori rd, rs, -1</code>	<code>1 ITYPE_pure64_RISCV_XORI rs1_val (-1)</code>
<code>mv rd, rs</code>	<code>addi rd, rs, 0</code>	<code>1 ITYPE_pure64_RISCV_ADDI rs1_val 0</code>

Table 1. Example of RISC-V pseudo-instructions, their aliased base instruction instance and the denotational semantics assigned to them in `riscv`. The semantics are derived via the official RISC-V assembly programming manual.

4.1.4 Implementation. `riscv` is implemented using Lean-MLIR, a heavily dependently typed software project that allows implementing SSA-based, MLIR-style dialects in the Lean ITP. A dialect implementer provides a set of operations and types that are bundled into a *Dialect* structure using the framework. For such a dialect structure, Lean-MLIR defines a set of typeclasses that must be instantiated, which subsequently allows leveraging dialect generic tooling.

Each dialect operation in `riscv` mirrors a RISC-V ISA instruction, which is implemented as an operation, where the source and destination register contents are operands and result. To implement the instruction types specified in the RISC-V ISA, we use instruction attributes. For example, immediate values are represented as instruction attributes, mirroring assembly instruction encodings, where the immediate is specified as part of the instruction bit pattern. Register values are explicitly modeled as input operands to the operation. For example, the R-type add operation is implemented with no attributes but two input operands, while the I-typed `slli` has a single input operand and a 6-bit bitvector attribute for its immediate. Both instructions are inhabitants of the inductive `Op` type, which is defined in Lean-MLIR to specify the set of dialect operations (Figure 18).

```

1 inductive Op
2   | add
3   | slli (shamt : BitVec 6)
4   \...\ the remaining set of ISA operations

```

Fig. 18. Each RISC-V ISA operation in `riscv` is modeled as an inhabitant of the Lean-MLIR `Op` type. An operand is specified with its instruction attributes, e.g., an immediate as in `slli`.

Note that a dialect implementation involves two abstraction layers: the dialect types and operations, and their denotation into Lean. The abstraction implemented by the dialect operations and types is conceptually unconstrained and defined by the developer. However, to reason about dialect objects, they must be denoted using an underlying Lean native type. For example, the type system of `riscv` contains `riscv.reg`, which is denoted using the `BitVec 64` type in Lean (Figure 19). Similarly, we leverage the functions extracted from Sail to denote `riscv` ISA dialect operations (Figure 4.1.4). To access the underlying denotation, the Lean-MLIR framework provides a function implemented for the `DialectDenote` type class.

```

491 1
492 2 instance : TyDenote Ty where
493 3   toType := fun
494 4     | Ty.riscvreg => BitVec 64

```

Fig. 19. Type denotation for `riscv`'s `riscv.reg` type which abstracts a bitvector of 64-bit width implemented as `BitVec 64` using Lean's native bitvector type.

```

499 1 instance : DialectDenote (RV64) where
500 2   denote
501 3   | .add, regs, _ => RTYPE_pure64_RISCV_ADD (regs.getN 1 (by simp [DialectSignature.sig, signature])) (regs.getN 0 (by simp [
502   DialectSignature.sig, signature]))

```

Fig. 20. Denoting an add operation using to `DialectDenote` typeclass in Lean-MLIR. Therefore, passing a function, `RTYPE_pure64_RISCV_ADD`, capturing the semantics of add and providing the two input operands in the operands list `regs` into the function. To provide correctness guarantees, we must prove that the read access to input operands is within bounds.

To allow writing dialect programs directly into the Lean frontend, we implement parsing functionality, which hooks into the MLIR syntax parser provided by Lean-MLIR. A program gets pre-parsed using our dialect-specific parser, which allows us to specify the syntax of our operations, e.g., using the instructions mnemonics as defined in the [ISA](#). The framework parser then generates an MLIR AST, which it serializes into a Lean dependent type. The Lean type represents intrinsically typed basic blocks named `Com`, which represents a sequence of `Exprs`, each of which models an individual SSA operation. Hence, for `riscv`, an `Expr` is typed over the [ISA](#) operation, its return type, namely a `riscv.reg`, its pure effect, as well as the input argument types. The approach of implementing `riscv` based on authoritative semantics using Lean-MLIR is adaptable to any [ISA](#) and its formal semantics.

We describe the example workflow of extending `riscv` with a single instruction.

- Add the instruction, together with its attributes, to the inductive `Op` type, and introduce its operand type as a constructor of the inductive `Ty` type ([Figure 21a](#)).
- Define the operation signature, specifying the input and output operand types of the dialect instruction ([Figure 21b](#) & [Figure 22a](#)).
- Formalize the semantic denotation of the operation via `DialectDenote`, expressed in terms of its inputs and attributes ([Figure 22b](#)).
- Extend syntax support for the dialect operation by implementing the dialect-specific parser ([Figure 22c](#)).
- Implement any additional typeclass instances as required, such as pretty-printing support via `toString`.

```

531 1 inductive Op
532 2   | add
533 3   | slli (shamt : BitVec 6)
534 4   | li : (val : BitVec 64) -> Op

```

(a) Adding a constructor for the `li` operation. The `li` constructor requires a 64-bit immediate to define the immediate to be loaded.

```

531 1 def Op.sig : Op -> List Ty
532 2   | .add => [Ty.riscvreg, Ty.riscvreg]
533 3   | .slli (_shamt : BitVec 6) => [Ty.riscvreg]
534 4   | .li _ => []

```

(b) `li` simply loads an immediate, which is an operation attribute. Therefore `li` requires no register input operands and accordingly the input signature is empty.

```

540 1 def Op.outTy : Op -> Ty
541 2 | .add => Ty.riscvreg
542 3 | .slli _ => Ty.riscvreg
543 4 | .li _ => Ty.riscvreg
544
545 1 instance : DialectDenote (RV64) where
546 2   denote
547 3   | .li imm, _, _ => imm

```

(a) The return type signature of a `li` consist of the `riscv.reg` type as it loads an immediate to be written into the destination register.

```

548 1 match opStx.name with
549 2 | "li" => do
550 3   let some att := opStx.attrs.getAttr "imm"
551 4   | throw <| .generic s!"no attribute in li, need to
552   specify immediate {repr opStx}"
553 5 match att with
554 6 | .int val ty =>
555 7   let opTy@(.riscvreg) ← mkTy ty
556 8   return (.pure, opTy, (
557 9     .li (val),
558 10    by
559 11      simp [DialectSignature.outTy, signature],
560 12      by constructor,
561 13      .nil,
562 14      .nil
563 15    ))

```

(c) Code fragment out of the `riscv` dialect parser that defines how an `Expr` is created by parsing the `li` dialect operation. The instruction attribute is extracted and passed into the constructor of the `Op` constructor for `li`. Error messages are thrown in case an unexpected type is provided.

Fig. 22. Adding support for the `li` pseudo-instruction in `riscv` leveraging the Lean-MLIR framework.

4.2 For Stepwise Lowering And Verification: A Hybrid IR

We introduce `riscv` which serves as an intermediate abstraction for reasoning, unlike conventional IRs designed for optimization, with semantics directly integrated in its operations such that any lowering can be validated, given a source program and its semantics. Compilers typically lower either in a single step or progressively, as in MLIR. Progressive lowering distributes transformations across abstraction levels, allowing different parts of a program to be transformed incrementally. While commonly motivated by optimization, we argue that multi-level IRs also benefits verification. Each single, local lowering step can be proven correct, and the proofs can be composed into an end-to-end correctness proof of the entire lowering. Hence, we repurpose the progressive structure for modular verification and implement the multi-level `llvmAndriscv` dialect, which embeds both the `llvm` and `riscv` dialects.

4.2.1 Motivation. The implementation of the `llvmAndriscv` hybrid IR as a tool to implement verified instruction selection is motivated by the following considerations:

- **Modular Proof Engineering:** A hybrid IR supports progressive lowerings from the `llvm` dialect to the `riscv` dialect. We explored an alternative approach in which a lowering was performed as a big-step transformation between two disjoint IRs. This strategy imposed a high proof-engineering burden as the entire transformation had to be verified at once, and even local instruction changes required reformulating the whole proof. In contrast, a stepwise transformation enables the verification of each individual instruction lowering and allows instruction selection to be modeled, performed, and proven progressively.

- *Compiler Practises*: A hybrid IR mirrors practices in realistic compiler backends and their instruction selectors. LLVM internally employs the *SelectionDAG* as an IR between LLVM IR and an IR containing machine instructions (MIR). In this DAG-based representation, LLVM IR operations are incrementally transformed into target-specific nodes. GlobalSel, the modern instruction selection framework in LLVM, also progresses from LLVM IR over a hybrid machine IR into a target-specific representation [6]. Such spirits are captured by our hybrid IR.
- *Tooling*: Embedding the `llvm` and `riscv` dialects as abstraction levels within a single hybrid IR enables the reuse of verified, dependently-typed infrastructure provided by Lean-MLIR. Modeling two abstraction levels within a unified IR emulates a single IR to such tools while still supporting cross-dialect lowerings.

4.2.2 Design. A multi-level IR supports multiple abstraction layers by implementing the instruction sets of several dialects, hence `llvmAndRiscv` combines operations from both the `llvm` and `riscv` dialects and unifies their types within a single type system. Using the hybrid `llvmAndRiscv` dialect, different parts of the program can reside at different abstraction levels (Figure 23). A technique to maintain type consistency when different dialects are intermixed is unrealized conversion casts, introduced by MLIR. The casts reflect a change in the static type of the value but have no semantic effect at runtime. Converting a value `x` from type `i64` to `riscv.reg` does not modify the concrete value present at runtime. The MLIR specification explicitly states: “This operation should not be attributed any special representational or execution semantics, and is generally only intended to be used to satisfy the temporary intermixing of type systems.” Hence, when a LLVM IR operand modelled in `llvmAndRiscv` is used inside a lowered RISC-V instruction also in `llvmAndRiscv`, a temporary cast is inserted to preserve type correctness. When the program is fully lowered into a single instruction set the casts are redundant and removed as only one type system remains.

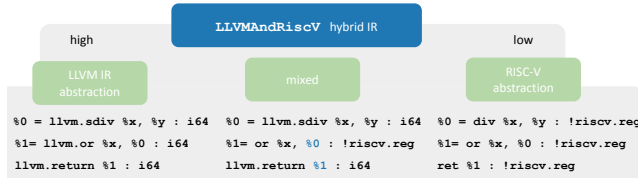


Fig. 23. **llvmAndRiscv hybrid IR**: Our hybrid IR supports multiple abstraction layers by embedding both the `llvm` and `riscv` dialects within a single unified representation. Operands requiring casts for type correctness are marked in blue. Namely, during progressive lowering, when the dialect instruction sets are mixed, conversion casts are needed to satisfy type correctness at the dialect boundaries. They are removed when the program is fully transformed into the target dialect, `riscv` in our case.

4.2.3 Implementation. Our hybrid IR is implemented using Lean-MLIR, however, with a focus on reusing the previously introduced dialects. Our approach can be adapted to implement a multi-level IR using any Lean-MLIR based dialects.

Types and Operations. : `llvmAndRiscv` supports the operations of both the `llvm` and `riscv` IRs. Additionally, the hybrid IR supports unrealized conversion casts, operations representing a cast from LLVM IR to `riscv` or vice-versa, parameterized by the bit-width they cast to/from. The type system of `llvmAndRiscv` includes the types of both dialects by means of `llvm` and `riscv` wrappers, each encapsulating a type from the respective dialect (Figure 24). For example, the `.riscv` type in the hybrid IR indicates that it is a type stemming from the `riscv` type system encapsulating the only `riscv` type, `!riscv.reg`.

```

638
639
640
641 1 inductive Ty where
642 2   | llvm : LLVM.Ty -> Ty
643 3   | riscv : RISCv64.RV64.Ty -> Ty
644

```

Fig. 24. Type system of the hybrid dialect

```

1 inductive Op where
2   | llvm : LLVM.Op -> Op
3   | riscv : RISCv64.RV64.Op -> Op
4   | castRiscv : Nat -> Op
5   | castLLVM : Nat -> Op

```

Fig. 25. Operation definitions in the hybrid dialect. The `llvm` and `riscv` constructors wrap operations from their respective dialects. The unrealized conversion casts are parameterized by a bitwidth, specifying conversions between bit-vector types of differing dialects.

Figure 26 shows a program modeled in the hybrid dialect, with the first part in `llvm` and the second in `riscv` 1. To correctly intermix types between dialects, we insert conversion casts 2, following MLIR’s naming.

```

655 1 ^entry (%a: i64, %b: i64):
656 2   %0 = llvm.icmp.eq %a, %b : i64
657 3   %1 = "builtin.unrealized_conversion_cast" (%0) : (i1) -> (!riscv.reg) 2
658 4   %2 = add %1, %1 : !riscv.reg 1
659 5   %3 = "builtin.unrealized_conversion_cast" (%2) : (!riscv.reg) -> (i1)
660 6   llvm.return %3 : i1

```

Fig. 26. A function written in `llvmAndRiscv` that contains both LLVM IR and RISC-V ISA instructions, and thus spans multiple levels of abstraction. To allow the intermixing of type systems, conversion casts, modeled as operations, are inserted at dialect boundaries (e.g., SSA values `%1` and `%3`).

Semantics. The semantics of operations and types in `llvmAndRiscv` dialect are defined by delegating to the `DialectDenote` and `TyDenote` instances of the respective source dialects by pattern matching and forwarding to the corresponding `TyDenote` instance. Similarly, the `DialectDenote` instance for the hybrid dialect uses pattern matching to extract the underlying operation, either LLVM or RISC-V, and delegates its interpretation to the original dialect. For example, a hybrid operation such as `.riscv add` is unwrapped in the native RISC-V `add` operation, and similarly, `.llvm sdiv` is unwrapped to `sdiv` to be then handled by the LLVM dialect. The unwrapping is implemented by helper functions (`llvmArgsFromHybrid` and `riscvArgsFromHybrid`) that define a transformation, e.g., from `LLVMRiscv.Ty.llvm` back to the original dialect types in `llvm` or `riscv` respectively.

```

678
679 1
680 2 instance : TyDenote LLVMPlusRiscv.Ty where
681 3   toType := fun
682 4     | .llvm llvmTy => TyDenote.toType llvmTy
683 5     | .riscv riscvTy => TyDenote.toType riscvTy
684

```

Fig. 27. Type system of the hybrid dialect

```

1 instance : DialectDenote (LLVMPlusRiscv) where
2   denote
3     | .llvm (llvmOp), args, .nil =>
4       DialectDenote.denote llvmOp (llvmArgsFromHybrid args) .
5         nil
6     | .riscv (riscvOp), args, .nil =>
7       DialectDenote.denote riscvOp (riscvArgsFromHybrid args) .
8         nil

```

Fig. 28. Operand definitions in the hybrid dialect

Conversion Casts. Recall that `riscv` has a simple type system consisting of a single `riscv.reg` type denoted as a `BitVec 64`. In contrast, the `llvm` dialect either operates over poison values or concrete values of the form `Option (BitVec w)`. Casting from `riscv` to `llvm` wraps a `riscv.reg` value in an `Option`, treating it as a non-poison LLVM value. In the reverse direction, casting from LLVM to `riscv` extracts the bitvector from the `Option` in case of `Option some` or assigns a default value in the case of poison modeled as `Option none`. Defaulting to a concrete value in case of **Undefined Behavior (UB)** is a semantic refinement (Section 4.3). The casts temporarily align the type systems, and will be resolved when fully lowered.

Syntax and Parsing. The syntax of the hybrid **IR** remains consistent with the generic MLIR syntax as implemented for the individual, underlying dialects. To maximize reuse of the existing infrastructure, we adopt staged parsing. When parsing a program in the hybrid dialect, each operation is first checked to be a cast and else attempted to be parsed as an `llvm` instruction. If that fails, parsing falls back to interpreting it as a RISC-V instruction. If both attempts fail, a parse error is raised. This layered strategy mirrors the hybrid **IRs** modular abstraction layers and reuses the parsing logic of its underlying dialects.

```

1  match llvmParse with
2  | .ok (eff, ty, expr) => do
3    let v <- transformExprLLVM expr
4    return (eff, .llvm ty, v)
5  | .error (..) => do
6    let (eff, ty, expr) <- RiscvMkExpr.mkExpr (ctxTransformToRiscV Γ) opStx (<-read)
7    let v <- transformExprRISCv expr
8    return (eff, .riscv ty, v)

```

Fig. 29. Using a staged parsing approach to parse the hybrid IR. We first try to parse an operation as `llvm` before trying to parse it as an RISC-V ISA instruction. If both fail, we throw an error.

Figure 29 shows a snippet of the staged parsing process after ruling out cast operations. The parser first attempts to interpret the operation using the `llvm` dialect. If successful (`.ok`), the parsed expression is lifted into the hybrid dialect `llvmAndRiscv` using `transformExprLLVM`, which produces a well-typed expression in the hybrid dialect given a well-typed expression in the source dialect. If `llvm` parsing fails, we fall back to the `riscv` parser and apply an analogous transformation over RISC-V expressions.

4.3 Correctness Statement

A hybrid **IR** as `llvmAndRiscv` allows for progressive lowering between dialects and, due to embedded semantics, also reasoning about the lowering. However, when is a lowering performed by a compiler, such as instruction selection, considered correct? In general, what is the specification of a correct compiler?

4.4 Refinement

When compiling a program, there is a relationship between the behavior of the original program and the resulting transformed program, which could require being an equivalence relation. However, languages like LLVM IR model **UB** that do not have a single defined behavior and a target architecture like a RISC-V processor execute instructions over concrete values. As a result, lowerings must resolve **UB** to concrete behavior supported by the processor. Requiring equivalence would be too strong, given that there is not always an equivalence, e.g., for a poison value, on hardware. The criterion for the correctness of a compiler is that the resulting program exhibits a subset of the behavior of the original program, captured as *refinements*, leading us to our correctness claim:

For all source programs S and compiler-generated code C , if the instruction selector, applied to the source S , selects the instructions in code C , without reporting an error, then the observable behavior of C refines the allowed observable behaviors of S , where S is a program using LLVM IR instructions and C is a program using 64-bit RISC-V ISA instructions.

Notice that in case all operands are concrete, refinement simplifies to equivalence. However, a compiler transformation is not reversible, when the lowered program permits only a subset of the original program’s behavior, e.g., when concretizing poison to a default value of 0. A LLVM add instruction, which resolves to a poison value on signed or unsigned overflow, is lowered into a RISC-V hardware addition, which always produces a concrete result (Figure 4.4). In the no-overflow case, both instructions have identical value semantics. On overflow, the RISC-V implementation is computed modulo the register width of 64, refining poison to a concrete value. We ensure that every transformation in our work is reasoned up to semantic refinement.

```
%2 = llvm.add %0, %1 overflow<nsw,nuw> : i64
```

Fig. 30. Original LLVM IR function written in `llvm`, can return poison on overflow or if an input operand is poison.

```
%2 = add %0, %1 : !riscv.reg
```

Fig. 31. Refining the `llvm` add operation into a `riscv` add operation which always yields a concrete value.

5 A Library Of Verified Instruction Selection Patterns

IRs, as introduced in Section 4, are intermediate compilation targets that abstract the program within the compiler, facilitating tasks such as optimization or, as `riscv` and `llvmAndRiscv`, reasoning about program semantics. In LLVM, transformations are implemented as rewrites over an IR and entire passes obtain their effect through the cumulative power of individual rewrites, e.g., `InstCombine`, a middle-end peephole optimizer, one of LLVM’s most important passes [4]. A rewrite matches a code sequence and transforms it into a lower-level representation (lowering) or into one that improves a performance metric, such as runtime or code size (optimization). Instruction Selectors as LLVM’s `SelectionDAG` [11] or `GlobalISel`² [17], are implemented to perform lowering rewrites that refine IR semantics, with UB, into machine code, with concrete value semantics, and to apply optimizing patterns during lowering. We extract and formalize a subset of such instruction selection (ISel) rewrites and verify them in Lean.

We obtain ISel rewrite rules from the opaque LLVM RISC-V backend and verify them up to semantic refinement. The resulting verified rewrites form a library of transformation patterns, which form the foundation of the verified LLVM IR-to-RISC-V instruction selector developed in this work. To avoid a high manual proof burden, a common stigma of verification projects, we design a Lean proof-automation tactic that discharges the refinement proof for LLVM ISel rules and allows to scale the collection of verified rewrites by supporting new realistic compiler patterns with minimal manual effort.

Set Up. We focus on `SelectionDAG` selection patterns over the LLVM integer types $t \in \{i1, i8, i16, i32, i64\}$, as prominent in LLVM’s `SelectionDAG` for RISC-V³. These types subsequently define the LLVM

²<https://llvm.org/docs/GlobalISel/index.html>

³In the `RISCVISelLowering.cpp` file within LLVM’s RISC-V target directory, legal types are registered using `addRegisterClass` and `setOperationAction`, where the number specifies the bit width. These types are handled by the instruction selector and are therefore considered legal. The listed types are multiples of eight and powers of two.

IR types supported by our instruction selector. Note that an instruction selection framework does not lower all **IR** types, but only legalized ones. A legal type is one that the instruction selector can process and does not necessarily need to be natively supported by the target machine, e.g., in LLVM, it is only required that the backend is able to convert a legal type into a native machine type. By providing lower patterns for our legal types, we make the supported cases explicit. An alternative approach, used in GlobalISel, is to assume all types are legal and instead specify only exception patterns. LLVM’s `i250` type requires legalization, such as being split into multiple `i64` values, unless the instruction selector directly supports its lowering. The set of legal operations in this thesis is the arithmetic fragment of LLVM IR as implemented in `l1vm` operating over legal types.

Action	Input	Transformed Code	Transformation
legal	<code>srem i32</code>	<code>srem i32</code>	legal, no legalization action
illegal	<code>srem i14</code>	<code>srem i16</code>	requires prior legalization
illegal	<code>fadd</code>	–	illegal operation, not supported in the LLVM dialect

Table 2. Assumption of legalization: the legal types are `i1`, `i8`, `i16`, `i32`, and `i64`. Legal operations are those in the LLVM dialect, while all others are assumed to have been emulated using legal ones.

Legal LLVM IR Arithmetic Operations:

- Bitwise Binary Ops: `shl`, `lshr`, `ashr`, `and`, `or`, `xor`
- Binary Ops⁴: `add`, `sub`, `mul`, `udiv`, `sdiv`, `urem`, `srem`
- Conversion Ops: `trunc`, `zext`, `sext`
- Others: `icmp`, `select`, `const`

5.1 Extracting Selection Patterns From LLVM

To extract LLVM **ISel** rewrites we use LLVM’s comprehensive suite of developer tools. The granularity of information varies by pass and tool, which leads us to employ two complementary approaches.

First, we derive instruction selection patterns by relying on `l1c`, LLVM’s static compiler [17] targeting a RISC-V 64-bit architecture with support for the M and B standard extensions (`l1c -O0 -march=riscv64 -mattr=+m,+b`). Each **IR** instruction is compiled at `-O0`, and the IR-to-ISA mapping is extracted from the assembly output. Although `l1c` invokes the full RISC-V backend, the invariant that each **IR** operation is lowered to an **ISA** instruction via the instruction selector still holds, allowing us to extract the selection patterns directly from its output (Figure 32 & Figure 33). Next, we compile each **IR** instruction across optimization levels up to `-O3` and observe that instruction selection remains consistent, despite LLVM using `FastISel` [6], a restricted subset of the `SelectionDAG` machinery, at `-O0`. The consistency of the instruction mapping across optimization levels suggests that the identified mapping forms a foundational rewrite of the LLVM RISC-V instruction selector.

To further validate our pattern extraction, we inspect the `SelectionDAG` for each instruction lowering using LLVM’s debug build (`-debug -only=isel-view-dag-combine1-dags`) (Figure 35). This view isolates the work of the instruction selector from the rest of the backend. From it, we extract the index of the applied pattern in the LLVM source code, thereby identifying the exact pattern used to match each **IR** instruction.

⁴The difference between bitwise and general binary operations is that bitwise operations are very efficient and can be strength reduced from other instructions [22]

```

834
835 1  define i64 @addnsw(i64 %a, i64 %b) {
836 2  entry:
837 3      %sum = add nsw i64 %a, %b
838 4      ret i64 %sum
839 5  }

```

Fig. 32. LLVM function performing a single i64 addition with the signed overflow flag (nsw) set which we subsequently compile via llc.

```

844
845 1  icmp_sgt:
846 2  # %bb.0:
847 3      slt a0, a1, a0
848 4      ret

```

Fig. 34. LLVM's signed greater-than comparison gets selected as the RISC-V slt instruction. Here, a0 holds the first function argument and a1 the second.

```

851 1  Selected selection DAG: %bb.0 'icmp_sgt:'
852 2  SelectionDAG has 9 nodes:
853 3      t0: ch,glue = EntryToken
854 4      t4: i64,ch = CopyFromReg t0, Register:i64 %1
855 5      t2: i64,ch = CopyFromReg t0, Register:i64 %0
856 6      t15: i64 = SLT t4, t2
857 7      t9: ch,glue = CopyToReg t0, Register:i64 x10,t15,t10 : ch = PseudoRETRegister : i64x10, t9, t9:1

```

Fig. 35. Textual description of the Selection DAG for the lowering of LLVM's greater-than comparison onto slt. The SelectionDAG allows us to exactly trace the work performed by LLVM's legacy instruction selector.

```

861
862 1  define i64 @zext16To64(i16 %a) {
863 2  entry:
864 3      %0 = zext i16 %a to i64
865 4      ret i64 %0
866 5  }

```

Fig. 36. LLVM function that performs a single greater-than comparison on the two input operands.

```

1  addnsw:
2  # %bb.0:
3      add a0, a0, a1
4      ret

```

Fig. 33. The add ISA instruction is generated from LLVM IR's add operation with the nsw overflow flag set. a1 refers to the second function argument, while a0 holds the first argument. The assembly shown is the direct output of llc.

```

1  zext16To64:
2  # %bb.0:
3      zext.h a0, a0
4      ret

```

Fig. 37. LLVM's sign-extensions of i16 to i64 is selected as the RISC-V zext.h ISA instruction.

Next, we determine whether a pattern is specialized by the RISC-V backend or provided by the target-independent backend infrastructure. LLVM backends rely on a large suite of shared tooling that can be extended with target-specific features. For example, common lowering patterns are supplied generically (Figure 38), and a backend only needs to bind the generic instruction definition, e.g., ADD, to its ISA counterpart, e.g., RISC-V (add).

Whether obtaining the textual assembly or the DAG, our compilation unit is a single LLVM IR operation, encoded within an LLVM IR function (Figure 32 & Figure 36), such that each basic block contains only one instruction. This prevents optimizations that would otherwise combine instructions within a basic block and thereby interfere with extracting the canonical instruction selection pattern.

In total, we retrieve patterns for all our legal llvm operations and their corresponding legal types and extract the LLVM pattern index to inform our mapping, and determine whether a pattern is

```

1  def : PatGprGpr<add, ADD>;
2  def : PatGprSimm12<add, ADDI>;
3  def : PatGprGpr<sub, SUB>;
4  def : PatGprGpr<or, OR>;
5  def : PatGprGpr<shiftpw<riscv_srlw>, SRLW>;

```

Fig. 38. SelectionDAG patterns are either specified generically using opcodes shared across backends, and later instantiated for a specific backend implementation, or some patterns (e.g., shift operations) are specialized for particular targets.

generic (Figure 39). A generic pattern instantiated unsoundly by one backend is likely incorrect across others as well. We classify instruction selection patterns into three categories: identity (direct mapping between IR and ISA instructions), one-to-one (an IR instruction mapped to a different ISA instruction), and one-to-many (an IR instruction mapped to multiple ISA instructions).

Operation	SelectionDAG Nodes Index	Attributes	Category
shl	1071279	nuw, nsw	id (sll)
lshr	1060739	-	id (srl)
ashr	1064723	-	id (sra)
and	15221	-	id (and)
or	5649	disjoint	id (or)
xor	122013	-	id (xor)
add	41652	nsw, nuw	id (and)
sub	99721	nsw, nuw	id (sub)
mul	132846	nsw, nuw	id (mul)
udiv	1228621	exact	id (divu)
sdiv	1226105	exact	id (div)
urem	1231137	-	id (remu)
srem	1068609	-	id (rem)
trunc	-	-	one-to-one (no-op)
zext	14836	-	one-to-one (andi)
sext	depends on types	-	depends on types
icmp ugt	1088297	-	one-to-one (sltu)
icmp uge	121779, 1088297	-	one-to-many (xori, sltu)
icmp ult	1088297	-	id (sltu)
icmp ule	121779, 1088297	-	one-to-many (xori, sltu)
icmp sgt	1088285	-	one-to-one (slt)
icmp sge	121779, 1088285	-	one-to-many (xori, slt)
icmp slt	1088285	-	id (slt)
icmp sle	121779, 1088285	-	one-to-one (xori, slt)
icmp eq	1088189	-	one-to-many (xor, sltu)
icmp ne	1088205	-	one-to-many (xor, sltu)
select	depends	-	one-to-many (andi, bnez)

Fig. 39. The fragment of SelectionDAG patterns shown is extracted from LLVM’s RISC-V backend for the legal types supported in 11vm. Each LLVM IR instruction, together with its attributes, is lowered using an indexed DAG pattern. Instructions that are lowered using several assembly instructions rely on several patterns, indicated by multiple indexes. trunc does not have a lowering pattern, as its internal representation is a no-op that doesn’t require lowering. To categorize the patterns, we record their mapping type and indicate the corresponding ISA instruction. For example, the zext LLVM IR instruction is lowered to a RISC-V no-op, since zero extension is not explicitly executed by the processor.

Of our two approaches, 11c is practical and reveals whether selected instructions remain consistent across optimization levels, while SelectionDAG exposes the exact patterns defined in LLVM and whether they are generic across backends. SelectionDAG further lets us trace the matching procedure, which lets us mirror LLVM’s matching order when implementing our verified selector. Unlike SelectionDAG, 11c’s assembly output contains pseudo-instructions introduced by assembler aliasing, e.g., seqz (Figure 41& Figure 42). LLVM’s instruction selector generally emits native ISA instructions (apart from framework instructions such as CopyToReg); however, before producing the final assembly specific instruction instances are aliased with declarative pseudo-instructions. Although aliasing is not an *Isel* rewrite, it is a code transformation that affects the selected instructions

in the textual assembly output and has been leveraged in translation validation, e.g., in [3]. Hence, we incorporate them into our instruction lowering pattern library as verified pseudo-instruction rules.

```

936                                     1  # %bb.0:
937                                     2      xor a0, a0, a1
938 1  # %bb.0:                          3      seqz a0, a0
939 2      xor a0, a0, a1                4      ret
940 3      sltiu a0, 1
941 4      ret

```

Fig. 40. Lowering pattern of LLVM IR's `icmp eq` instruction according to the SelectionDAG pattern not using pseudo-instructions.

Fig. 41. `Seqz` (set if equal to zero) is a pseudo-instruction, hence not supported by a concrete RISC-V machine target, but serves as a more declarative alias for `sltiu` in the textual assembly. It is the output of `llc` when lowering LLVM IR's `icmp eq` instruction.

```

946 1  def : InstAlias<"seqz rd,rs", (SLTIU GPR:
947      rd,GPR :rs, 1)>;

```

Fig. 42. Instruction aliasing pattern for the specific instance of `sltiu` with the constant 1.

5.2 LLVM ISel Patterns In Lean

After extracting LLVM's instruction selection patterns over the integer fragment of LLVM IR, we formalize each in Lean to validate up to semantic refinement.

An LLVM instruction selection pattern ($A \rightarrow \text{asm}$) is modeled as a tuple (A, asm, p) , where A is the IR instruction, asm its lowered replacement, and p a refinement proof certifying that A is refined by asm under all possible valuations. Hence, we mechanize it as a verified peephole rewrite that crosses the abstraction boundary between LLVM IR and the target ISA, and requires a refinement proof certificate. When modeling the instruction to be matched, we mechanize its operands either as free variables in a context or as constants, depending on the extracted pattern. The LLVM instruction `add i64 x1 x2` is formalized as an instruction in a context of two free `i64` variables (`x1`, `x2`). Moreover, in SSA-based IRs, each instruction produces an SSA value, hence a mechanization of an instruction match pattern also contains the type of the value returned by the instruction. For example, the LLVM instruction `add i64` produces a value of type `i64`. Given free variables (interpreted as program block arguments), the instruction, and its return type, we mechanize an **ISel** find-pattern that captures the structure required for a well-defined match during instruction selection. Thus, for a program text P and a find-pattern A , we locate instances of A in P by matching its free variables, instructions, and result type. We build on the IR models implemented in Section 4, which give us instruction models with a trustworthy semantic foundation.

The replace-pattern (asm) within a LLVM rewrite ranges from a single ISA instruction to an entire sequence of assembly operations. We mechanize the target sequence by identifying its free variables, its return type, and the computation it encodes, which can contain several instructions. In the LLVM IR `add <nsw, nuw>` example, a possible lowering pattern is the RISC-V ISA `add` instruction, which is represented by two free variables, the ISA instruction `add`, and its resulting return type.

After formalizing the match A and replacing sequences asm , we must provide a proof p , which connects the match- and replace-pattern via a refinement argument. p certifies that A is refined

by asm under all valuations, which are concrete instantiations of the free variables modelling all possible invocations of the matched LLVM instruction. Generally, providing p elevates a LLVM ISEL pattern from a best-effort rewrite to a verified refinement step.

The formalization of our rewrites in Lean is similar to Bhat et Al’s [4] formalization of LLVM IR middle-end InstCombine peephole rewrites, however, adapt to the mappings implemented by LLVM’s instruction selection.

In Lean, we mechanize the tuple (A, asm, p) using a structure consisting of a left-hand side (lhs **1**), representing the source pattern A , a right side (rhs **2**) associated with the lowering asm and a correct field (correct **3**) capturing the refinement proof p (Figure 43).

```

1 structure LLVMPeepholeRewriteRefine (w : Nat) (Γ : Ctxt Ty) where
2   lhs : Com LLVMAndRV64 4 Γ .pure (Ty.llvm (.bitvec w)) 1
3   rhs : Com LLVMAndRV64 Γ .pure (Ty.llvm (.bitvec w)) 2
4   correct : ∀ V,
5     PoisonOr.IsRefinedBy (lhs.denote V) (rhs.denote V) 3

```

Fig. 43. Formalization of a LLVM instruction selection pattern as a verified peephole rewrite that consists of a match and replace-pattern as well as correct field capturing the refinement proof. Proving the rewrite results in constructing a term of the type stating that the rhs refines the lhs.

```

1 def ISEL_add_nuw_nsw_flag : LLVMPeepholeRewriteRefine 64 [Ty.llvm (.bitvec 64), Ty.llvm (.bitvec 64)] where
2   lhs := add_llvm_nsw_nuw_flags
3   rhs := add_riscv
4   correct := .....

```

Fig. 44. Concrete instantiation of a LLVM rewrite in Lean. The rewrite is named ISEL_add_nuw_nsw_flag and captures the lowering of an LLVM add with flag attributes into an assembly add, each specified by an instruction sequence in lhs and rhs not displayed in the figure.

Our encoding consists of a context Γ that captures free variables in the lhs match-pattern. We then declare an instruction selection pattern **some_isel_pattern** which has the type `LLVMPeepholeRewriteRefine (w : Nat) (Γ : Ctxt Ty)` and is dependently typed over the hybrid IR (`llvmAndRiscv`) containing `llvm` and the RISC-V **ISA** **4**. w defines the bit-width of the final instruction we are matching with the pattern. The lhs and rhs of the selection pattern are written in Lean using MLIR syntax, facilitating the definition of a pattern. Lean’s elaborator de-sugars the MLIR syntax based rewrites into inhabitants of the type `Com`. Recall that `Com` captures an instruction sequence with free variables in Γ and a pure effect. Our `Com` instances define that the matched instruction is of a LLVM type. The proof p then certifies the rewrite by requiring an inhabitant of a term that is dependently typed over the lhs and rhs. Concretely, providing a term of the dependent type $\forall V, \text{PoisonOr.IsRefinedBy } (\text{lhs.denote } V) (\text{rhs.denote } V)$, proves the rewrite via the Curry–Howard correspondence, where programs are proofs and propositions are types. The proof p can be discharged either manually using tactics provided by Lean-MLIR [4] or automatically via proof automation built on these tactics, as described in the next section. Regardless of the strategy, the underlying proof obligation lies in the bitvector domain.

Figure 46 shows an LLVM rewrite pattern encoded in Lean with explicit casts to reconcile type systems. Since Lean-MLIR patterns are restricted to a single dialect, casts are inserted at


```
10301      %1 = llvm.add %arg0, %arg1 overflow<nsw,nuw> : i64
```

```
1031
```

Fig. 45. Lhs of the formalization of the formalized addition ISel pattern of LLVM in Lean. `%arg0` and `%arg1` are the free variables in the rewrite used to identify a well-formed match.

```
10341 %0 = "builtin.unrealized_conversion_cast"(%arg0) : (i64) -> (!riscv.reg)
```

```
10352 %1 = "builtin.unrealized_conversion_cast" (%arg1) : (i64) -> (!riscv.reg)
```

```
10363 %2 = add %arg0, %arg1 : !riscv.reg
```

```
10364 %res = "builtin.unrealized_conversion_cast" (%2) : (!riscv.reg) -> (i64)
```

```
1037
```

Fig. 46. rhs of LLVM's addition SelectionDAG rewrite for the RISC-V target. Explicit casts at the boundaries enforce Lean-MLIR's type constraints and simulate the rewrite as an inline assembly rewrite in LLVM IR.

```
1040
```

```
1041
```

dialect boundaries to satisfy type constraints when formalizing LLVM lowerings across dialects. Conceptually, the rewrite behaves like an inline assembly expansion, where each instruction is replaced by a snippet and casts ensure temporary type alignment. Repeated application yields a program expressed entirely in assembly. Figure 47 illustrates this process for LLVM's `icmp eq` over `i64`.

```
1047
```

```
1048
```

```
1049 1 def icmp_eq_llvm_i64 : Com LLVMPlusRiscV ([.llvm (.bitvec 64), .llvm (.bitvec 64)])
1050 2   .pure (.llvm (.bitvec 1)) :=
1051 3   [LV] {
1052 4     ^entry (%lhs: i64, %rhs: i64):
1053 5       %1 = llvm.icmp.eq %lhs, %rhs : i64
1054 6       llvm.return %1 : i1
1055 7   }
1056 8
1057 9 def icmp_eq_riscv_i64 :=
1058 10 [LV] {
1059 11   ^entry (%lhs: i64, %rhs: i64):
1060 12     %lhsr = "builtin.unrealized_conversion_cast"(%lhs) : (i64) -> (!riscv.reg)
1061 13     %rhshr = "builtin.unrealized_conversion_cast"(%rhs) : (i64) -> (!riscv.reg)
1062 14     %0 = xor %lhsr, %rhshr : !riscv.reg
1063 15     %1 = sltiu %0, 1 : !riscv.reg
1064 16     %2 = "builtin.unrealized_conversion_cast"(%1) : (!riscv.reg) -> (i1)
1065 17     llvm.return %2 : i1
1066 18   }
1067 19 def icmp_eq_riscv_eq_llvm_i64 : LLVMPeepholeRewriteRefine 1 [Ty.llvm (.bitvec 64), Ty.llvm (.
1068 20   bitvec 64)] :=
1069   {lhs:= icmp_eq_llvm_i64, rhs:= icmp_eq_riscv_i64}
```

```
1063
```

Fig. 47. Formalizing the LLVM's ISel pattern for `icmp eq` in Lean. The lhs is the LLVM IR's `icmp eq` to be matched, while the rhs specifies the RISC-V ISA replace pattern. By leveraging the formal IR models of LLVM IR and RISC-V in Lean, we can mechanically reason about the soundness of the LLVM rewrite. The proof is automatically discharged in Lean, as will be introduced in Section 6

```
1069
```

```
1070
```

ISel Patterns For Lowering Constants In Lean: A restriction we encountered during our work is that LLVM IR constants in `llvm` are formalized as operands that encode their value as an attribute. Hence, each constant requires an individual lowering pattern as Lean-MLIR doesn't support attribute values as free variables. Therefore, we implement a pattern for each constant in the range of $[-50, 50]$ as well as for selected constants encountered. Future work can implement the parameterization over attributes to improve the formalization of IRs in Lean, which is not the target of this thesis.

```
1077
```

```
1078
```

5.3 Proof and Verification in Lean

After specifying an LLVM instruction selection pattern in Lean, we provide a proof p that lowering the lhs to the rhs is sound. Our formalization represents patterns in a hybrid IR combining LLVM IR with the RISC-V ISA, enabling reasoning about both sides through their semantic denotations as embedded the dialect models in Lean (Section 4). Correctness is established via semantic refinement, accounting for LLVM’s undefined behaviors, e.g., poison, against the ISA’s concrete value semantics. We next outline the proof technique used to certify patterns, which we automate in the next section (Section 6). Contrasting rewrite-testing heuristics that only leave us to believe a pattern is correct, a mechanization like ours guarantees a sound rewrite.

In summary, to prove correctness we extract a proof obligation from the formalized rewrite, which reduces to the underlying domain of bitvectors and can be discharged using tactics and bitvector-compatible solvers. However, a purely bitvector goal state arises only in the case of equality reasoning, when both the IR and the ISA operate on concrete values. When the LLVM IR pattern is instantiated with poison, the reasoning extends beyond pure bitvector reasoning.

To prove the rewrite, we construct a term typed as the proposition that the LLVM IR instruction is refined by the specified RISC-V lowering, for all instantiations of the free variables: $\forall V$, $\text{PoisonOr.IsRefinedBy } (\text{lhs.denote } V) (\text{rhs.denote } V)$, where lhs and rhs are the match-and-replace pattern provided by the developer as described in Section 5.2. This requires demonstrating that the LLVM IR instruction in the lhs permits exactly the set of behaviors that are valid for the RISC-V ISA operation in the rhs, informally captured as $(\text{rhs.behavior} \subset \text{lhs.behavior})$. rhs.behavior contains the semantics of a RISC-V instruction sequence as specified by the Sail RISC-V ISA semantics. Figure 48 shows the instantiated proposition for proving the lowering of LLVM IR’s `i64 icmp eq`. To prove a rewrite, we generally proceed by case splitting on the possible

```

1041  ∀ (V : ([Ty.llvm (.bitvec 64), Ty.llvm (.bitvec 64)]).Valuation) 1,
1052  PoisonOr.IsRefinedBy (icmp_eq_llvm_i64.denote V) (icmp_eq_riscv_i64.denote V)

```

Fig. 48. Proof goal for the `ISel` pattern of LLVM IR’s `icmp eq`, which compares two integers for equality. The proof requires showing that for all instantiations of the two free variables of type `.bitvec 64`, which can either be poison or a bitvector, the LLVM IR patterns is refined by the RISC-V lowering. By calling `denote`, we extract the denotation of the lhs and rhs.

values of the free variables in the valuation. In Figure 48, we reason over all combinations of poison and non-poison instantiations for the two free LLVM IR variables 1. Notice that the proof obligation is obtained after removing the dialect encoding of a rewrite such that MLIR nor SSA are featured anymore, e.g., `expand Com` and `Expr`. Aside from a RISC-V-specific `simp_tactic`, which unfolds ISA semantics in the RISC-V dialect, the simplifications are performed via dialect-agnostic tactics and stem from prior work by Bhat et al. [4].

```

1120 1 ⊢ BitVec.ofBool (x == x) \lhs =
1121 2 BitVec.signExtend 1
1121 3 (BitVec.setWidth 64
1122 4 (BitVec.ofBool ((BitVec.signExtend 64 x).xor (BitVec.signExtend 64 x) < BitVec.signExtend 64 1#12))) \rhs

```

Fig. 49. Goal state when proving the LLVM IR `icmp eq` pattern after removing the dialect encoding and extracting the denotation of the rewrite rhs and lhs. The proof goal captures the subgoal when all operands are concrete, whereas the poison cases are contained in different subgoals.

After simplification, the remaining proof goal is expressed over native Lean types (Figure 49). This goal is discharged by case distinction on the free variables in the LLVM IR match pattern. Note that undefined behavior (UB) appears only in the lhs of the rewrite, which necessitates case reasoning over the free variable argument occurring in the lhs.

- ≥ 1 Poison Operand: In the case the lhs matches on an instruction with a poison-typed free variable used as an input argument, poison will taint the return type. An ISA operation always returns a concrete value, and concrete refines poison. Therefore a rewrite of poison to a concrete value is proven via refinement. We show that $\text{rhs.behavior} \subset \text{lhs.behavior}$, where the subset relation is strict. In our supported LLVM IR fragment, the select operation is an exception; it may still produce a concrete value even if one of its inputs is poison, provided that the poisoned input is not selected. Reasoning about select can proceed via equality even if poison is present.
- Fully Concrete Operands: When all operands are concrete, refinement reduces to equality. In this case, we prove that both the original and the lowered patterns evaluate to the same bitvector formulas that encode the semantics of the IR and ISA operations with concrete input operands, informally expressed as $\text{rhs.behavior} = \text{lhs.behavior}$.

In the rewrite of LLVM IR's icmp eq operation with two input operands modeled as free variables, lowered to the RISC-V sltiu instruction, we obtain four subgoals. These correspond to: (1) the case of two poison operands, where poison is refined by any concrete assembly result ($\text{poison} \sqsubset \text{some concrete value}$); (2–3) the two asymmetric cases with one poison operand, where the poisoned input taints the result, again yielding $\text{poison} \sqsubset \text{some concrete value}$; and (4) the fully concrete case, where we reason about the equality of the resulting bitvector formulas ($\text{concrete} = \text{concrete}$).

To automate this reasoning pattern for each rewrite with an eye towards scalability, we incorporate the proof intuition outlined above into our Lean formalization of rewrites. In this setup, the developer's task is limited to specifying the candidate rewrite, while the proof p , certifying the correctness of the pattern, is derived automatically.

6 Proof Automation For Rewrite Patterns

Verification provides strong correctness guarantees but is known to “[...] require substantial user effort”[26], particularly in interactive environments such as *ITPs*. As Meng et al.[26] write, “an obvious step is to gain the best of both worlds [...]”, hence after formalizing the rewrite pattern in Lean, we aim for automation and shift the proof burden from the user to an end-to-end tactic.

Our automation strategy reduces proof obligations to domains with strong solver support. We first simplify and rewrite framework code to expose obligations in Lean's native types (BitVec, Nat, Int), and ensure that dialect semantics align with the solver's target domain. Lean provides a verified bit-blasting solver, which we leverage by adapting the Sail-based RISC-V model (riscv) to bitvector semantics, naturally reflecting register values (Section 6.1). This preprocessing enables an end-to-end proof hammer that automatically discharges rewrite proofs (Section 6.2).

6.1 “Bit-Blastable” RISC-V Semantics

The model of LLVM IR semantics used in this work is formalized over the Option (BitVec w) type as implemented in [4], while the Sail RISC-V model provides ISA denotations over bitvectors but frequently introduces type conversions for readability (Figure 50). We adapt this semantics to be automation-friendly by assigning ISA operations directly to bit-blastable interpretations, as hinted in Section 4.

Conversions in the Sail RISC-V model, such as interpreting a `BitVec` as a `Nat` or `Int` lift values into unbounded domains, breaking compatibility with solver techniques like bit-blasting that assume uniform fixed-width bitvectors [14, 16]. Figure 51 shows part of the `mul` semantics, where `BitVec`-

```

11811 \\explicit signed representation, not bit-blastable
11812 BitVec.toInt rs1_val
11813
11814 \\bit-blastable
11815 rs1_val

```

Fig. 50. A `BitVec`-to-`Int` conversion of a bitvector (`rs1_val`). Sail-RISC-V heavily relies on such to facilitate the readability of the mechanized ISA specification; however, an integer definition hinders bit-blasting automation, which requires a fixed-width bitvector.

to-`Nat`/`Int` conversions express an operand sign via integer interpretation, reflecting architectural intent rather than the bitvector computation. To enable automated ISA verification, we reformulate these semantics for the operations in our extracted LLVM pattern set. The verified rewrite removes type conversions, making `riscv` semantics directly bit-blastable and proof obligations solvable by `bv_decide` (Figure 52).

```

1195
1196 def execute_MUL (rs2 : regidx) (rs1 : regidx) (rd : regidx) (mul_op : mul_op) : SailM Retired := do
1197   let rs1_val ← do (rX_bits rs1)
1198   let rs2_val ← do (rX_bits rs2)
1199   let rs1_int : Int :=
1200     if mul_op.signed_rs1
1201       then (BitVec.toInt rs1_val)
1202       else (BitVec.toNat rs1_val)
1203   ...

```

Fig. 51. The Sail RISC-V `mul` instruction semantics convert the bitvector value according to the sign of the operation to natural numbers or integers. While this facilitates to interpretation of the bitvector value, it shifts the value into an unbounded domain, which in turn hinders solvers from benefiting from the finite, enumerable domain of a fixed-width vector.

```

1208 def MUL_pure64_ttf_bv (rs1_val : BitVec 64) (rs2_val : BitVec 64) : BitVec 64 :=
1209   BitVec.extractLsb 127 64 (((BitVec.signExtend 129 rs1_val) * (BitVec.zeroExtend 129 rs2_val)))

```

Fig. 52. The Sail RISC-V semantics of a signed multiplication, returning the high bits, are reformulated into a pure, bit-blastable bitvector definition over the domain of a 64-bit vector.

Each transformation is backed by a formal Lean proof (Figure 53), ensuring a faithful implementation of the RISC-V ISA semantics throughout. In total, we converted 78% of the RISC-V ISA supported in `riscv` through such rewrites, including the proof, where some required up to 33 tactic invocations⁵. The main challenges arose from handling overflow behavior and reconciling the mapping between integer/natural number and bitvector operations. In particular, Sail-RISC-V defines division and remainder via truncated division (`tdiv/tmod`), which required us to show that integer `tdiv` aligns with bitvector `sdiv` and `udiv`. Moreover, bitvectors operate modulo their width. Due to natural numbers and integers, the Sail RISC-V semantics enforce bounds via programmatic checks. For signed division, `IntMin` is specified as the result of an overflow; in contrast,

⁵We count tactic invocations by lines: a single call to, e.g., a `simp` tactic is counted as one invocation, even if it internally applies multiple tactics.

bitvector division wraps on overflow, which requires us to reason about the equivalence of these two behaviors (hard-coded constant vs. wrap-around). This required proofs with up to 31 tactic invocations, e.g., in the overflow case of `IntMin` divided by `-1`. This generally raises questions about the inherent design of Sail models (Figure 53). While they provide mechanized and trustworthy ISA specifications, their structure makes automation laborious, which we improve by once performing the labour and incorporating bit-blastable semantics into a RISC-V model in Lean. During our

```

12331 theorem SIGNED_pure64_DIV_eq_SIGNED_pure64_DIV_bv (rs2_val : BitVec 64) (rs1_val : BitVec 64) :
12342   SIGNED_pure64_DIV rs2_val rs1_val = SIGNED_pure64_DIV_bv rs2_val rs1_val := by
12353   unfold SIGNED_pure64_DIV SIGNED_pure64_DIV_bv
12364   rw [extractLsb'_ofInt_eq_ofInt (h := by simp)]
12375   by_cases h1 : rs2_val = 0#64
12386   case pos => simp [h1]
12397   case neg =>
12408     have h' : rs2_val.toInt ≠ 0 := by
12419       have h1 := (BitVec.toInt_ne).mpr h1
124210      exact h1
124311     apply BitVec.eq_of_toInt_eq
124412     simp only [Int.reduceSub, h', reduceIte, Int.reduceNeg, BitVec.toInt_ofInt,
124513       h1, Int.reducePow, Int.reduceSub, Int.reduceNeg, Nat.reducePow, BitVec.toInt_sdiv]
124614     by_cases h : rs1_val = .intMin _
124715     ^ rs2_val = -1#64
124816     [...]

```

Fig. 53. Beginning of the mechanized proof of the conversion of integer/natural signed division into bitvector division. Mapping integer/natural operations onto the bitvector domain is not always a straightforward mapping. However, the mapping onto purely bitvector semantics enables proof automation via optimized bitvector solvers, which leads to questioning the inherent design of the Sail RISC-V mechanization, as integers and naturals hinder automation.

rewriting procedure, we extracted several generic Lean bitvector theorems applicable beyond the setting of our work (Figure 54).

```

12541 theorem sshiftRight_eq_setWidth_extractLsb_signExtend (w : Nat) {n : Nat} (x : BitVec w) :
12552   x.sshiftRight n =
12563   ((x.signExtend (w + n)).extractLsb (w - 1 + n) n).setWidth w := by
12574
12585   ext i hi
12596   simp only [getElem_sshiftRight, getElem_setWidth, getLsbD_extract, add_tsub_cancel_right,
12607     show i ≤ w - 1 by omega, decide_true, getLsbD_signExtend, Bool.true_and]
12618   by_cases hni : (n + i) < w <=> simp [hni] <=> omega

```

Fig. 54. As part of reformulating our semantics, we established a generic lemma capturing that right shifting a bit vector of width w by n positions, is equal to first sign extending the bitvector by n bits and then extracting the left-most w bits.

Building on the now bit-blastable RISC-V semantics, we next present a tactic that leverages such to automatically discharge the end-to-end proofs for LLVM ISel rewrites.

6.2 A Proof Hammer For Instruction Selection Patterns

"Obstacle to the wider use of proof assistants is the low level of automation" [], which we counteract by refining our LLVM rewrite formalization in Lean to automatically derive the rewrite proof, whenever the obligation falls within the underlying solver's domain. At the core lies the verified `bv_decide` bit-blasting solver, such that every proof is certified by a *verified* verification tool, contrasting to reliance on unverified SMT solvers with a history of bugs, commonly used in

compiler verification projects, as in [3]. Recall how an instruction lowering pattern is registered as a structure of source and target instruction sequences and a correct field capturing the proposition to be proven as a Curry-Howard isomorphism as the proof object type (Figure 55).

We tune the structure with a default auto-tactic that automatically invokes a `simp_lowering` tactic to minimize the `ISel` pattern encoding, followed by `bv_decide` to solve the remaining bitvector formula. `Simp_lowering` uses the Lean simplifier, which examines a designated set of lemmas and left-to-right rewrite rules (*the simp set*) exhaustively toward solving the current proof goal. We invoke the simplifier with different lemma sets and strengthen existing tactics, e.g., by extending the `simp` set, to compose the `simp_lowering` tactic ¹. More generally, we exploit the flexibility of Lean’s ecosystem to dynamically adapt proof-automation tactics, enabling a single tactic to scale across a range of rewrites. `simp_lowering` eliminates Lean-MLIR internal data structures, e.g., `Com`, `Expr`, `HVector`, `Valuation` and simplifies the `lhs` pattern in the `llvm` dialect as implemented in [4]. Next, it internally invokes `simp_riscv`, our tactic custom to the RISC-V ISA, which retrieves the embedded, bit-blastable semantics derived in the previous subsection. After simplification, we perform case analysis based on poison and non-poison operand types, mechanizing the proof structure outlined in Section 6.1. We therefore systematically reduce the goal such that the obtained sub-cases are directly discharged during simplification, e.g. `poison` refined to `concrete`, or subsequently solved by the `bv_decide` procedure ². The tactics combined discharge the entire rewrite proof obligation. We separate the invocation of `bv_decide` and `simp_lowering` within the rewrite structure such that in case the `bv_decide` invocation fails, `simp_lowering` can be used independently to simplify a proof goal, such as unfold definitions.

```

structure LLVMPeepholeRewriteRefine (w : Nat) (Γ : Ctxt Ty) where
  lhs : Com LLVMPlusRiscV Γ .pure (Ty.llvm (.bitvec w))
  rhs : Com LLVMPlusRiscV Γ .pure (Ty.llvm (.bitvec w))
  correct : ∀ V,
    PoisonOr.IsRefinedBy (lhs.denote V) (rhs.denote V) := by
      simp_lowering 1 <=> bv_decide 2

```

Fig. 55. The `LLVMPeepholeRewriteRefine` structure contains the rewriting pattern and automatically falls back to auto-tactics to derive the correctness proof. `Simp_lowering` tries to convert the proof goal into a statement that can be discharged by the subsequent call to `bv_decide`.

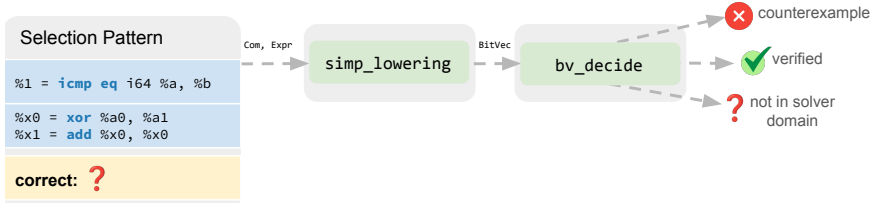


Fig. 56. A candidate selection pattern is formalized in Lean, and its proof is attempted automatically using `simp_lowering` followed by an invocation of `bv_decide`. If the pattern is sound and lies within the supported bitvector domain of `bv_decide`, it is certified. For an unsound pattern, a counterexample is produced, or the solver reports that the pattern could not be decided.

A previously manual proof for lowering LLVM’s shift-left instruction with two overflow attributes into a RISC-V shift-left (Figure 57) now requires no manual effort. (1) The developer specifies

only the source and target sequences (Figure 58). (2) Auto-tactics attempt the proof, returning a counterexample if the pattern is unsound. (3) If the goal lies outside the solver’s domain, Lean reports this, and the user proceeds manually. (4) On success, the pattern is fully verified for refinement via a proof, leaving no room for bugs.

```

1 def llvm_shl_lower_riscv: LLVMPeepholeRewriteRefine 64 [Ty.llvm (.bitvec 64), Ty.llvm (.bitvec 64)] where
2   lhs := shl_llvm
3   rhs := shl_riscv
4   correct := by
5     unfold shl_llvm shl_riscv
6     simp_peephole
7     simp_alive_undef
8     simp_riscv
9     simp_alive_ops
10    simp_alive_case_bash
11    intro x x'
12    split
13    case value.value.isTrue htt =>
14      simp
15    case value.value.isFalse hff =>
16      simp only [Nat.cast_ofNat, BitVec.ofNat_eq_ofNat, ge_iff_le, BitVec.not_le] at hff
17      simp only [BitVec.shiftLeft_eq', Nat.sub_zero, Nat.reduceAdd, PoisonOr.toOption_getSome,
18        BitVec.setWidth_eq, BitVec.extractLsb_toNat, Nat.shiftRight_zero, tsub_zero,
19        Nat.reducePow, BitVec.signExtend_eq, PoisonOr.value_isRefinedBy_value,
20        InstCombine.bv_isRefinedBy_iff]
21      rw [Nat.mod_eq_of_lt (a:= x'.toNat) (b:= 64)]
22      bv_omega

```

Fig. 57. Manually proving the instruction lowering pattern for the `shl` instruction. The proof requires several tactic invocations to minimize the framework overhead in Lean-MLIR and proceeds by case distinction on the type of the operands, whose intuition we described in Section 5.3.

```

1 def llvm_shl_lower_riscv_nsw_nuw: LLVMPeepholeRewriteRefine 64 [Ty.llvm (.bitvec 64), Ty.llvm (.bitvec 64)] where
2   lhs := shl_llvm_nsw_nuw
3   rhs := shl_riscv

```

Fig. 58. No manual proof effort is required, the proof is discharged by the `simp_lowering` and `bv_decide` tactics. The user only needs to provide the pattern via `lhs` and `rhs`, the `correct` field and its proof obligation are automatically derived by Lean due to the default auto-tactic embedded into a rewrite formalization.

Our proof automation discharges all previously manual refinement proofs for our LLVM ISel patterns, which on average required 11 tactics each prior to automation (Figure 60). This enables verification of new patterns in our language fragment at scale by eliminating the manual proof bottleneck. We obtain a library of LLVM instruction selection rules, each verified up to refinement, yielding trustworthy IR-to-ISA mappings.

7 Peephole Optimization

Using LLVM parlance, SelectionDAG-based instruction selection includes DAGCombiner routines, optimization passes that perform instruction merging (*combines*) as each DAG node is visited during pattern-matching and subsequent selection. In Section 5.1 we extracted the individual, per-instruction SelectionDAG patterns and automatically verified them for refinement in Lean (Section 5.2). These lowering patterns are isolated from DAG combining, as we implemented one instruction per basic block, leaving no additional instructions for combining. However, within a basic block, SelectionDAG aims for optimized instruction selection, a design criticized as opaque

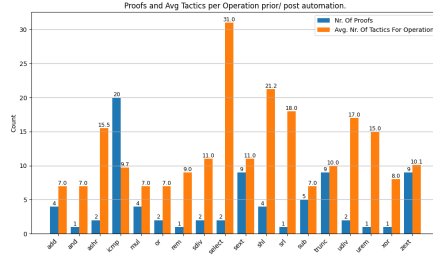


Fig. 59. This statistic shows the number of proofs per operation class and the average number of tactic calls needed per operation class. Before automating the proofs each required at least 7 tactic invocations by us. After automation, no user-guided tactic call is required. Lean infers the proof automatically, given it lies within the domain solvable by the underlying `bv_decide` SMT solver.

and complex, one of the reasons being this coupling. Indeed, bugs frequently arise from the DAGCombiner routines, e.g., due to merging instructions which wrongly taint each other with poison, recently seen in [33].

In contrast, the new GlobalSel selector improves the monolithic design by providing a regular machine pipeline, e.g., separating optimization into distinct passes. This aligns with the progressive and modular transformation paradigm seen in frameworks like MLIR. GlobalSel implements a shared library of peephole rewrites used in three combiner optimization passes, each pass supporting a curated subset of the rules, depending on its placement within the backend. We extract (Section 7.1), mechanize and verify (Section 7.2) GlobalSel optimizations for integration into our verified rewrite library for optimized but verified instruction selection (Section 8). Since GlobalSel is the modern instruction selector under active development, we claim that optimizations it already incorporates are of particular significance. Documentation in the codebase, e.g., “Add more combines”⁶ [17] indicate the ongoing effort; nevertheless the LLVM community converged on a set of rewrite rules for RISC-V, highlighting them as core rewrites of a LLVM RISC-V instruction selector and RISC-V compiler backend.

By formally verifying the GlobalSel combines for the LLVM IR arithmetic fragment and re-implementing them in our selector, this chapter contributes toward a trustworthy GlobalSel’s combiner foundation.

We exclude optimization patterns over floating-point operations, pointers, or with reliance on LLVM’s Known Bits [17] dataflow analysis. The first two types are not yet supported by our framework, but can be added in future work⁷. The latter stems from limitations in Lean-MLIR and reflects a broader design decision about the sophistication of peephole rewrites. In addition, we focus on patterns targeting 64-bit RISC-V. While LLVM instantiates patterns for RISC-V processors with varying general-purpose register widths, we restrict to 64-bit types in RISC-V and LLVM. Extending the approach to other widths, e.g., 32-bit, involves replicating the same pattern formalization. The underlying methodology remains unchanged and benefits from the proof automation demonstrated in Section 7.2.

⁶<https://github.com/llvm/llvm-project> in `llvm/Target/GlobalSel/Combine.td`

⁷Recent work has introduced floating-point reasoning support in the Lean theorem prover, providing a foundation for extending our approach to floating-point optimizations.

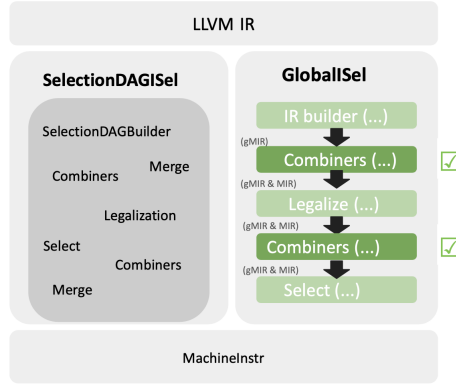


Fig. 60. The two LLVM instruction selectors, SelectionDAG and GlobalISel, take different approaches to optimization placement and modularity. SelectionDAG operates on a DAG-based intermediate representation and performs all transformations in a single, monolithic pass. In contrast, GlobalISel derives a generic MIR (gMIR) from LLVM IR and incrementally refines it into target-specific machine instructions (MIR) through a series of modular passes. Along the way, it operates over a hybrid IR that mixes generic and target-specific constructs. Our work adopts the pipeline architecture of GlobalISel, leveraging its modularity to extract, formally verify, and re-implement its combiner optimizations.

7.1 LLVM’s GlobalISel Optimizing Rewrites

In GlobalISel’s pipeline, optimizations are factored out into combiner passes at a fixed location in the backend; however, a developer can, by desire, insert custom passes into the GlobalISel pipeline, and for this reuse rewrites of a collection of target-independent combine patterns. These combine encode the general logic behind a rewrite and are instantiated for the specific target. Hence, GlobalISel rewrite rules are shared across passes and across processor targets.

GlobalISel Pattern Extraction. We extract and mechanize optimizations from two of the three default RISC-V GlobalISel combiner passes, the combiner used at `-O0` (`RISCV00PreLegalizerCombiner`) and the postlegalizer combiner (`RISCVPostLegalizerCombiner`). The former is applied even with no optimization, while the latter is explicitly designated as optimizing and is the intended location for adding new rewrites. Although some optimizations run before ISel, they can already manipulate target-specific opcodes. GlobalISel progressively lowers a generic IR to target-specific forms, and by invariant all opcodes are target-specific after ISel. This design permits specialization and lowering prior to ISel, however less common. Each GlobalISel pass is assembled by hand from a library of proposed rewrites (Figure 61).

```
def RISCV00PreLegalizerCombiner: GICombiner<
  "RISCV00PreLegalizerCombinerImpl", [optnone_combines]> {
}
```

Fig. 61. Definition of the combiner pass for the RISC-V GlobalISel instruction selector at `-O0`. The pass is constructed from rewriting patterns in `optnone_combines`, a list initializer of patterns and pattern groups, that are instantiated for the RISC-V target and applied during instruction selection at `-O0`.

Figure 62 shows a target-independent rewrite logic (Figure 63) implemented by the RISC-V backend and applied within the selection framework (Figure 64), resulting in optimized RISC-V instruction selection visible in the output assembly (Figure 65). The subtraction of a constant is

transformed into the addition of its negation within the instruction selection framework. We claim this leverages the base ISA’s support for `addi` (add immediate), enabling further optimization, while a corresponding `subi` is absent. Note that such rewriting is applied during ISel, even at the lowest optimization level, and cannot be disabled in LLVM.

```

14771 define i64 @sub_constant_lhs(i64 %a) {
14782   %0 = sub i64 %a, 1
14793   ret i64 %0
14804 }
```

Fig. 62. Original LLVM IR function that gets compiled using the zero optimization level, performing a subtraction of a 64-bit typed SSA variable with 1.

```

14851
14862   %3:_(s64) = G_CONSTANT i64 -1
14873   %2:_(s64) = G_ADD %0:_, %3:_
1488
```

Fig. 64. Optimized function by the GlobalISel RISCVO0PreLegalizerCombiner within the instruction selection framework. This figures shows the exported function (Figure 62) directly after the combiner is applied over the IR.

```

1      match (G_CONSTANT c, imm),
2          (G_SUB d, op1, c),
3          [{return matchCombineSubToAdd}]
```

Fig. 63. Core of the combiner pattern in the GlobalISel library of rewrites that matches on constant subtraction and rewrites it into an addition, enabling further optimizations depending on the ISA.

```

1  # %bb.0:
2      addi a0, a0, -1
3      ret
```

Fig. 65. The output assembly when compiling using -O0. Hence, even at the lowest optimization level, GlobalISel’s combines are invoked, making them a valuable target for verification and integration into our pipeline.

We extract these rewrites directly from the LLVM codebase (Figure 7.1), preserving LLVM’s organization into individual rules or groups of logically related rules (Figure 67). This yields a collection of optimizing rewrites from an instruction selection framework that is highly trusted in LLVM, as executed even at -O0. Still, they are empirically selected by balancing code size and compile time [6], while correctness is only implicitly assumed, making the verification in our Lean framework in the next section valuable. Our extracted patterns are listed in Figure 66 and Figure 67

PostLegalizer-Combines	#Patterns in GlobalISel	Mechanized
sub_to_add	1	100%
combines_for_ext_load	2	0% (memory)
redundant_and	1	100%
identity_combines	41	57% (memory)
shift_immed_chain	5	0% (dataflow)
commute_constant_to_rhs	29	45% (floating point)
simplify_neg_minmax	1	100%
Total	80	50%

Fig. 66. GlobalISel PostLegalizer-Combines that we extracted and verified are integrated into our certified instruction selector using the naming convention in the RISC-V backend. Each row represents a pattern or pattern set (#Patterns). These combiners are always run in the RISC-V GlobalISel selector except at -O0. We support all patterns within the arithmetic fragment, those who are not implement require either vector, floating point or pointer operations as indicated when we do not support 100%. However, even if a pattern doesn’t lie within our supported fragment, the rewrite rule logic often remains the same, just one a different data type, e.g., vector add instead of an integer addition.

O0PreLegalizer-Combines	#Patterns in GlobalSel	Mechanized
trivial_combines*	6	50% (floating point, memory)✓
not_cmp_fold	8	100%✓
others	5	0%(memory,vector)
Total	19	58%

Fig. 67. GlobalSel -O0 Prelegalizer-Combines extracted and mechanized in Lean. They are executed at zero optimization and composed into a pass, where each entry represents either a set of rewrites or a single rewrite. We focus on rewrites within the LLVM IR arithmetic fragment and the base ISA, including the M and B extensions, excluding load/store operations. The combines summarized by others operate over vectors, floating points or pointers. *contains a single rewrite applied only to instruction-selection helper instructions, such as a copy that moves values between registers, implemented as an intrinsic pseudo-op

7.2 Implementation and Proofs in Lean

To verify GlobalSel’s optimized selection rules, we implement the extracted set of rewrite patterns in a hybrid dialect. This multi-level dialect `llvmAndRiscv` (Section 4) enables the formalization of rewrite rules that combine high-level IR semantics (LLVM IR) with target-level instructions (the RISC-V ISA). Such a design directly reflects the hybrid nature of GlobalSel itself, which operates over both generic and target-specific constructs in its intermediate representations (gMIR and MIR). Our verification strategy for these optimizing patterns proceeds along two complementary tracks, both grounded in our IR models in Lean:

- **Generic Rewrite Patterns:** Patterns expressed using generic opcodes (denoted with the prefix `G` in LLVM) are formalized and verified with respect to the semantics of LLVM IR (Figure 68). This is justified by the structure of gMIR, which is a derived intermediate translation layer, capturing LLVM IR semantics, as confirmed in the IR translation passes in LLVM and in [6].
- **Target-Specific Rewrite Patterns:** Machine code optimizations implemented by RISC-V specific opcodes or mixes of `G_` (generic) and `RISCV_` opcodes (target-dependent) are certified against our authoritative, formal model of RISC-V ISA semantics (Figure 69).

```

lhs := [LV] {
  ^entry (%x: i64, %c: i1):
    %0 = llvm.select %c, %x, %x : i64
    llvm.return %0 : i64}
rhs := [LV] {
  ^entry (%x: i64, %c: i1):
    llvm.return %x : i64}

```

Fig. 68. Optimization that folds a select operation over the same operand into the operand itself. This is an optimization supported in the RISC-V GlobalSel post-legalization combiner and certified against the LLVM IR semantics implemented in Lean. The proof is deferred to the next section.

```

lhs := [LV] {
  ^entry (%x: !riscv.reg):
    %c = li (0) : !riscv.reg
    %0 = ror %x, %c : !riscv.reg
    ret %0 : !riscv.reg}
rhs := [LV] {
  ^entry (%x: !riscv.reg):
    ret %x : !riscv.reg}

```

Fig. 69. `right_identity_zero_pat_rotr` combiner in the RISC-V GlobalSel backend. We mechanize it using `riscv` in Lean and model its logic as a match (lhs)- and replace (rhs) peephole rewrite.

Similar to Section 5.2, when formalizing Isel lowering patterns, we use a Lean structure to model a dependent rewrite triple (lhs, rhs, p) , consisting of a find pattern, a replace pattern, and a correctness certificate. The lhs and rhs of each rewrite are formalized over either LLVM IR or the RISC-V abstraction, and subsequently embedded as Lean datatypes to reason within the ITP.

While in Section 5.2, we focused on lowering patterns, where lhs and rhs belong to different IRs, optimizing rewrites operate within the same IR over LLVM IR or ISA semantics. Hence, we introduce a second structure specialized for rewrites at the RISC-V ISA level (Figure 71), where the proof obligation is strengthened from refinement (Figure 70) to equality, given that ISA instruction operands are concrete bitvectors (1 Figure 71).

Therefore, a GlobalSel rule over generic opcodes is mechanized as a match-replace pair (lhs and rhs) with a refinement proof, e.g., folding a select instruction over a single operand into the operand itself constitutes such a GlobalSel pattern (?). At the ISA level, rewrites such as eliminating a zero right-rotation in RISC-V assembly are likewise captured in GlobalSel patterns, modeled over the RISC-V dialect and requiring an equality proof between the denotation of rhs and lhs.

```

1 LLVM IR lowering proof obligation.
2 correct : ∀ V,
3   PoisonOr.IsRefinedBy (lhs.denote V) (rhs.denote V)

```

Fig. 70. Proof obligation for an LLVM IR lowering or optimizing rewrite. GlobalSel uses target-generic opcodes (prefixed with G_), derived from LLVM IR translations, which we model and verify against the LLVM IR semantics.

```

1 structure RISCVPeepholeRewrite (Γ : Ctxt Ty) where
2   lhs : Com LLVMAndRV64 Γ .pure (Ty.riscv (.bv))
3   rhs : Com LLVMAndRV64 Γ .pure (Ty.riscv (.bv))
4   RISC-V ISA assembly rewrite proof obligation
5   correct : (lhs.denote) = (rhs.denote) 1

```

Fig. 71. Lean structure implemented to formalize GlobalSel rewrites that operate over machine-specific opcodes, which, in a RISC-V backend, correspond to RISC-V ISA assembly mnemonics. ISA-level rewrites are verified for equality, since refinement collapses to equality in the case of concrete values.

Formal Verification of Optimizing Rewrites From LLVM’s Instruction Selector. The proof structure for an optimizing rewrite follows the methodology outlined in ?. Although these rewrites are situated within an instruction selection framework, each rewrite remains within a single abstraction layer, and the proofs do not cross dialect boundaries. However, reasoning over LLVM IR rewrites requires refinement arguments, which fall back to the framework in ?. In contrast, rewriting over the target-dependent RISC-V IR is verified for semantic equality.

To avoid writing manual proofs, we leverage the `simp_lowering` and `bv_decide` proof automation tactics (Section 6.2). Recall that any dialect encoding is removed and therefore only a mathematical, Lean native proof obligation remains, which is either simplified by Lean’s library support, e.g., the simplifier, or discharged by an invocation to the verified `bv_decide` bit-blasting solver. We mechanize all extracted GlobalSel rewrites and automatically verify them to extend our certified pattern library at no additional proof burden. Moreover, this demonstrates that our proof automation is well-tuned for instruction selection patterns for LLVM IR’s arithmetic fragment and, more generally, for LLVM IR and RISC-V local peephole transformations.

The entire set of verified rewrites is finally composed into several list initializers, analogous to LLVM, which contains the rewrite collection and is used to construct a pass in our instruction selector. Such a combiner list is typed over the rewrite structure (Figure 72).

```

16181 def PostLegalizerCombiner_RISCV: List (Σ Γ, RISCVPeepholeRewrite Γ) :=
16192   RISCV_identity_combines ++
16203   RISCV_commute_int_constant_to_rhs ++
16214   RISCV_simplify_neg_minmax
16225
16236 def PostLegalizerCombiner_LLVMIR_64 : List (Σ Γ, LLVMPeepholeRewriteRefine 64 Γ) :=
16247   LLVMIR_sub_to_add ++ LLVMIR_redundant_add ++ LLVMIR_select_same_val
16258   ++ LLVMIR_identity_combines
1626

```

Fig. 72. Modeling GlobalSel Combiners in Lean. We verify the underlying pattern groups and compose the resulting collection of rewrite patterns into a combiner pass in Lean, which serves as the foundation for optimizing our instruction selector. Two distinct peephole structures, including the context of free variables of a rewrite, are employed to formalize the patterns together with their corresponding semantics in Lean.

8 The Instruction Selector

Effective and correct instruction selection is crucial for converting high-level languages to efficient and trustworthy machine code. Widely adopted programming languages such as C++, Rust, Lean, and even Python trust LLVM’s backend and thereby its instruction selector to reliably translate a generic IR into high-performance assembly code. To reduce the trust base and push toward verified compiler development, we present a verified instruction selector for LLVM IR’s arithmetic fragment targeting a 64-bit RISC-V processor. The selector combines fully formally verified SelectionDAG instruction selection patterns (Section 5) with optimizing GlobalSel rewrites (Section 7) to improve selection quality and is extended with new patterns with little proof burden. We integrate the trustworthy selector as a certified pass into an otherwise unverified backend. The result is executable 64-bit RISC-V machine code generated through a compilation pipeline with verified instruction selection at its core.

We detail the implementation based on a verified pattern rewrite engine (Section 8.1) and the collection of certified selection rules (Section 8.2), and finally describe the integration into an end-to-end LLVM IR-RISC-V compilation flow (Section 9 & Section 9.1).

8.1 A Fully Verified Multi-Pattern Rewrite Engine

Instruction selection for an input program proceeds by matching a library of lowering patterns against the program until it is fully expressed in the target ISA. Accordingly, we require tooling that systematically applies instruction selection patterns to transform the input into its lower-level representation.

Lean-MLIR provides a fully verified peephole optimizer, which we leverage for peephole rewriting in its most general form, spanning multiple dialects and lowering across IR toward ISA. We pass the library of locally verified rewrite patterns to a verified rewriter, which acts as a matching engine, lifting local correctness of applying a single verified selection pattern toward the correctness of the whole lowering by applying several rewrites. We have proven each local ISel rewrite to refine program semantics (Section 5 & Section 7), and their composition yields a refining big-step transformation by invoking the verified rewriter.

However, the verified rewriter in Lean-MLIR serves only as a starting point for our instruction selector. Unlike the original rewriter, which applies a single peephole transformation, instruction selection requires:

- Applying an entire library of rewrite patterns iteratively.

- Applying rewrites to a fixed point, ensuring that all instructions are fully lowered into ISA opcodes.
- Applying verified rewrites under semantic refinement, rather than strict equality.

We extend the rewrite infrastructure in Lean-MLIR to address the requirements above and support the application of multiple rewrites. The implementation consists of two parts: (1) the rewriter, implemented as a function, and (2) its correctness proof, which constructs a term of the type asserting that the program’s denotation is preserved after applying the rewriter.

(1) *The multi-rewriter.* The extended rewriter processes a dependently typed list of patterns drawn from our verified library and iterates over the input program, attempting to match and apply rewrites at each program location. To guarantee termination, we introduce a fuel variable, initialized to the size of the input program. If the fuel reaches zero, the current state of the selected program is returned; otherwise, the rewriter proceeds to the next instruction for selection (Figure 73). At each program location, the rewriter folds the list of selection patterns over the instruction until a match is found or no patterns remain. Thereby, it invokes `rewritePeepholeAt`, which attempts a single rewrite at the given instruction, as provided by Lean-MLIR. The procedure resembles the way SelectionDAG performs matching attempts, as exposed by its traces.

(2) *Correctness.* The multi-rewriter is proven correct by constructing a term of type `(multiRewritePeephole fuel prs target).denote = target.denote`⁸, thereby showing that the denotation of the program is preserved under instruction selection. The proof proceeds by first establishing correctness of folding the pattern list over a single program location. In the base case, no pattern is available (the list is `.nil`), and the semantics are trivially preserved since no transformation occurs. In the inductive step, the next pattern in the list is considered. If the pattern applies, correctness follows from the correct field of the pattern structure (`lhs`, `rhs`, `correct`), which ensures semantic refinement. If the pattern does not apply, no rewrite is performed, and semantics are again preserved. This argument leverages the correctness lemma of the single-pattern rewriter for each individual rule. Building on this, we prove that folding over all patterns at all program locations preserves semantics (Figure 74). Finally, we establish a lemma showing that initiating the rewriting process at program location zero, which is the exact definition of the multi-rewriter (Figure 73, line 12), preserves the denotation of the entire program, yielding a term of the desired type (1, Figure 74).

We benefit from the SSA-based implementation of the original Lean-MLIR optimizer, which matches along the def-use chain [4]. The rewriter walks upward through the program text from a matched return value and attempts to match the remaining part of the pattern, without requiring straight-line code. Once a pattern is identified as a subprogram of the whole program, it can be matched, and the corresponding instruction is lowered into the ISA pattern.

The exact rewriting of a single instruction by the rewrite engine first matches the pattern (Figure 75), substitutes the uses of the matched value, and then applies **Dead Code Elimination (DCE)** to remove the initially matched, now dead, left hand-side (1 Figure 76 & Figure 77)⁹. Based on Lean-MLIR’s peephole rewriter and an extending induction proof, we have shown that if each pattern is a valid refinement, the instruction selector based on local peephole rewriting using a set of verified ISel patterns is refining, and thus preserves the specification of a correct compiler.

⁸fuel is the termination variable, set to the program size; prs is the list of verified selection rules; and target is the input program.

⁹DCE is implemented in LeanMLIR as a separate pass. First, this allows us to access DCE as an optimization outside of the rewrite engine. Secondly, implementing DCE within the rewriter would have been wrong for instructions that use several operands, which are still matched in a def-use chain of a following pattern.

1:36

```
17161 def multiRewritePeepholeAt (fuel : ℕ) (prs : List (Σ Γ, Σ ty, PeepholeRewrite d Γ ty))
17172   (ix : ℕ) (target : Com d Γ' eff t') : Com d Γ' eff t' :=
17183   match fuel with
17194   | 0 => target
17205   | fuel' + 1 =>
17216     let target' := prs.foldl (fun acc ⟨_Γ, _ty, pr⟩ => rewritePeepholeAt pr ix acc) target
17227     multiRewritePeepholeAt fuel' prs (ix + 1) target'
17238
17249 /-- rewrite with the list of peephole optimizations `prs` at the `target` program, running at most
172510 `fuel` steps starting at location 0. -/
172611
172712 def multiRewritePeephole (fuel : ℕ)
172813   (prs : List (Σ Γ, Σ ty, PeepholeRewrite d Γ ty)) (target : Com d Γ' eff t') : (Com d Γ' eff t') :=
172914   multiRewritePeepholeAt fuel prs 0 target
173015
```

Fig. 73. Implementation of the multi-rewrite rewrite engine that uses a fuel variable as well as the set of rewrites as a termination variable. `multiRewritePeephole` starts matching patterns over a program at the first instruction in the program and then subsequently calling `multiRewritePeepholeAt`, which performs a single matching step by folding the next rewrite to match over the program. It calls `rewritePeepholeAt`, which was provided by Lean-MLIR in its single rewrite engine.

```
1731
1732
17331 /- The proof that applying `rewritePeephole_go_multi` preserves the semantics of the target program
17342 to which the peephole rewrites get applied. -/
17353
17364 theorem denote_multiRewritePeepholeAt (fuel : ℕ)
17375   (prs : List (Σ Γ, Σ ty, PeepholeRewrite d Γ ty)) (ix : ℕ) (target : Com d Γ' eff t') :
17386   (multiRewritePeepholeAt fuel prs ix target).denote = target.denote := by
17397   induction fuel generalizing prs ix target
17408   case zero =>
17419     simp [multiRewritePeepholeAt]
174210   case succ hp =>
174311     simp [multiRewritePeepholeAt, hp] using denote_foldl_rewritePeepholeAt ..
174412
174513 /- The proof that `rewritePeephole_multi` is semantics preserving -/
174614
174715 theorem denote_multiRewritePeephole (fuel : ℕ)
174816   (prs : List (Σ Γ, Σ ty, PeepholeRewrite d Γ ty)) (target : Com d Γ' eff t') :
174917   (multiRewritePeephole fuel prs target).denote = target.denote 1 := by
175018   exact denote_multiRewritePeepholeAt ..
175119
```

Fig. 74. Proving that the denotation of the program is refined by applying rewrites via the multi-rewriter. First, we prove semantic-preservation of `denote_multiRewritePeepholeAt`, showing a single rewrite step correct, based on the correctness proof encoded into the rewrite itself (correct). Next, we use it as a lower-level lemma to prove the entire procedure correct in `denote_multiRewritePeephole`.

```
1752
17531 %1 = llvm.and %lhs, %rhs : i64
17542 llvm.return %1 : i64
1755
```

Fig. 75. When lowering the `and` instruction, the rewrite matches the corresponding rewrite. In this case, any pattern with the above program text on the left side will match.

8.2 Composing The Instruction Selector

Next, we determine how to assemble the rewrite collection and in what order patterns should be matched. This reduces to a local phase ordering problem, since the order of instruction selection can directly influence subsequent selection and optimization opportunities.

```

17651 %lhs = "builtin.unrealized_conversion_cast" (%lhs) : (i64) -> !riscv.reg
17662 %rhs = "builtin.unrealized_conversion_cast" (%rhs) : (i64) -> !riscv.reg
17673 %l0 = and %lhs, %rhs : !riscv.reg
17674 %l1 = "builtin.unrealized_conversion_cast" (%0) : (!riscv.reg) -> (i64)
17685 %1 = llvm.and %lhs, %rhs : i64 dead 1
17696 llvm.return %l1 : i64

```

Fig. 76. At the end of the matched def-use chain, the rewriter inlines the lowered instruction sequence and iterates forwards through the program replacing any use of the matched return value with its new SSA value. The initial instruction becomes dead.

```

17751 %lhs = "builtin.unrealized_conversion_cast" (%lhs) : (i64) -> !riscv.reg
17762 %rhs = "builtin.unrealized_conversion_cast" (%rhs) : (i64) -> !riscv.reg
17773 %l0 = and %lhs, %rhs : !riscv.reg
17774 %l1 = "builtin.unrealized_conversion_cast" (%0) : (!riscv.reg) -> i64
17785 llvm.return %l1 : i64

```

Fig. 77. After running dead code elimination, the old, matched instruction is removed, and the lowered pattern remains.

Different Instruction Lowering Phases. We assemble all rewrites into a list initializer in Lean, partitioned by type of lowering pattern: pseudo-instruction lowering, constant lowering, optimization patterns, and core arithmetic LLVM IR instruction lowering, each specialized to the supported integer type. The verified rewrite engine is invoked sequentially with the list of potential matches, interleaved with verified SSA-based optimizations. The lowering stages within our instruction selector are ordered as follows:

- Initial verified DCE
- Optional pseudo-instruction lowering
- Optional GlobalSel optimizations (Prelegalizer and Postlegalizer combines)
- Constant lowering
- LLVM IR arithmetic instruction selection
- Verified DCE
- Cast reconciliation
- Verified DCE
- Verified Common Subexpression Elimination (CSE)
- Optional GlobalSel optimizations (Prelegalizer and Postlegalizer combines)

At the start of instruction selection, and between lowering phases, we eliminate dead code, usually assumed to have been removed already by the optimizing middleend. Our dead code elimination is a fully verified optimization implemented in [4] for the Lean-MLIR optimizer. We then perform optional pseudo-operation lowering and GlobalSel optimizations, depending on the optimization flags with which the instruction selector is invoked. Next, constants are matched and lowered, followed by the entire LLVM IR arithmetic fragment, yielding a program representation containing only ISA opcodes and casts. Then casts are reconciled, following MLIR’s workflow, which are introduced during lowering for temporary type system alignment. Because selection proceeds at the per-instruction level, some instructions remain in LLVM IR while others are lowered, depending on the placement of their corresponding rewrite in matching machinery. The `unrealized_conversion_casts` prevent type clashes during lowering and are eliminated once a uniform type system is reached after full instruction selection, using a verified cast reconciliation pattern.

A Rewrite For Cast Reconciliation. The verified rewrite for reconciling cast folds away cyclic pairs of unrealized conversion casts, which are artifacts from progressively selecting instructions. After lowering a program fully into the RISC-V IR, the remaining casts are removed as a uniform RISC-V type system is reached and they become redundant (Figure 78). Our cast removing pattern is verified using the introduced (lhs, rhs, correct) Lean structure with a corresponding refinement proof showing that removing cyclic cast is a refinement. We have verified, for our case, what MLIR performs across its dialect translations.

```

18221 lhs := [LV] {
18222   ^entry (%arg: !riscv.reg):
18223     %0 = "builtin.unrealized_conversion_cast"(%arg) : (!riscv.reg) -> (i64)
18224     %1 = "builtin.unrealized_conversion_cast"(%0) : (i64) -> (!riscv.reg)
18225     ret %1 : !riscv.reg]]
18226 rhs := [LV] {
18227   ^entry (%arg: !riscv.reg):
18228     ret %arg : !riscv.reg]]

```

Fig. 78. Cast reconciling rewrite. A cyclic cast from !riscv.reg to the LLVM IR type i64 back to !riscv.reg is removed, as it reduces to a no-op.

```

18333 before:
18334   %add1 = add %lhsr, %rhsr : !riscv.reg
18335   %add1 = "builtin.unrealized_conversion_cast" (%add1) : (!riscv.reg) -> (i64)
18336   %add2 = "builtin.unrealized_conversion_cast" (%add1) : (i64) -> (!riscv.reg)
18337   %add3 = add %add2, %rhsr : !riscv.reg
18338 after:
18339   %add1 = add %lhsr, %rhsr : !riscv.reg
18340   %add3 = add %add1, %rhsr : !riscv.reg

```

Fig. 79. After progressively lowering onto the ISA, the cyclic cast can be removed. They are not required as the program has fully converged into SSA-based assembly and therefore no temporary intermixing of types is needed.

After cast reconciliation, we run DCE as well as CSE to polish the output program with verified optimizations. Finally, we support optional LLVM GlobalISel rewrites that optimize beyond dead-code elimination and common-subexpression substitution by performing LLVM peephole optimizations within dialect boundaries. Finally, if no error was thrown, we obtain a fully selected program.

9 Backend Integration

We integrate our verified instruction selector into a backend for the 64-bit RISC-V target. The intermediate output of our framework is the LLVM IR program lowered into the RISC-V model in Lean, where each selected instruction is proven correct. Since our IR models in Lean adopt MLIR syntax, the textual format of the selector output is expressed in generic MLIR syntax. We therefore employ an MLIR compilation flow to obtain a register-allocated program from our SSA ISA representation. Importantly, the register allocator doesn't affect our selected instructions, thereby preserving the guarantees of our certified instruction selection. Verifying the register allocation is left for future interesting work.

We pipe the IR output of our selector into the RISC-V xDSL backend [12], a compiler framework that reimplements MLIR in Python and integrates with MLIR developer tools. Before register allocation, we run a custom pass that parses the selector output and ensures syntactic alignment

```

18631 %2 = "riscv.or"(%1, %1) : (!riscv.reg, !riscv.reg) -> (!riscv.reg)
18642 %3 = "riscv.sub"(%2, %1) : (!riscv.reg, !riscv.reg) -> (!riscv.reg)
18653 %4 = "riscv.rem"(%3, %3) : (!riscv.reg, !riscv.reg) -> (!riscv.reg)
18654 %5 = "riscv.srl"(%1, %4) : (!riscv.reg, !riscv.reg) -> (!riscv.reg)

```

Fig. 80. The intermediate output of our instruction selector contains the selected RISC-V ISA operations in SSA form. By using MLIR’s generic syntax, we can directly reuse MLIR’s compilation toolkit to generate executable code.

with MLIR, including function headers. This pass adapts headers and footers and refines casts at dialect boundaries. The refinement argument is as follows: replacing a function header with poison and concrete values by a concrete header and footer constitutes a semantic refinement. We omit further details as the reasoning proceeds as in [Section 5](#). The pass is publicly available in our code base.

Register Allocation. To perform the conversion from SSA assembly to register-allocated RISC-V machine code, we leverage the unverified register allocator implemented by Lopoukhine et Al. [25] as part of their python-MLIR based custom RISC-V backend. We extend the register allocator for our ISA instructions, which are mostly supported already by the xDSL RISC-V dialect [12], which `riscv` models closely. However, we expand on this by adding support for 46 additional ISA extension instructions contained in `riscv`, and leverage MLIR’s uniform generic syntax to directly parse our lowered code into the register allocator.

Importantly, the xDSL register allocator is semantics-agnostic: it assumes that the input program, written in SSA form, has correct semantics and variable assignments. The sole responsibility of the allocator is to assign registers to variables according to the structure of the input program, which is abstracted by the types and the number of register operands each instruction requires. The exact, spill-free register allocation algorithm is detailed in [25].

We print the textual assembly after register allocation by providing the corresponding flags (`xdsl-opt -riscv-allocate-registers` and `-t -riscv-asm`), mirroring how `llc` in LLVM outputs its assembly. As [Figure 81](#) demonstrates, the register allocator only operates on the arguments to each operation and does not change the generated IR-to-ISA mapping nor any ISA instruction itself.

9.1 Interactive Lowering Via Command Line

Developers working with LLVM and MLIR know the optimizer `opt`, which lets them hand-pick passes and assemble a pass pipeline over the input code. This makes pipelines modular and extensible, and lifts the compiler toward a more interactive, customizable tool. To offer the same command-line interaction for our instruction selector, we implemented an `opt`-style executable in Lean that mirrors the LLVM optimizer within the ITP. Our tool parses `.mlir` files containing the MLIR LLVM IR dialect and, provided the file content lies within our supported LLVM IR fragment, runs our verified instruction selection over the input file. The output is the ISA intermediate representation, which contains only RISC-V instructions selected for the input LLVM IR program. Conceptually, `opt` bridges the gap between the Lean dialect formalizations and their concrete instantiation in a compiler pipeline. Our command-line tool can be invoked with the following flags directly in the Lean environment:

- `verbose`: Parses the input file into the LLVM dialect model in Lean and prints the ITP-native dialect representation, built from Lean data structures such as `Com` and `Expr`, to the command line. This exposes the Lean-internal encoding of the program and shows exactly how Lean reasons about a dialect leveraging dependent types.

```

1912      1 builtin.module {
1913      2   riscv_func.func @main(%0 : !riscv.reg) -> !riscv.reg {
1914      3     %1 = riscv.li 8 : !riscv.reg
1915      4     %2 = riscv.li 31 : !riscv.reg
1916      5     %3 = riscv.sra %0, %2 : (!riscv.reg, !riscv.reg) -> !riscv.reg
1917      6     %4 = riscv.and %3, %1 : (!riscv.reg, !riscv.reg) -> !riscv.reg
1918      7     %5 = riscv.add %4, %3 : (!riscv.reg, !riscv.reg) -> !riscv.reg
1919      8     riscv_func.return : !riscv.reg
1920      9   }
1921     10 }

```

(a) Preregister allocation: Our semantics-preserving RISC-V assembly IR code generated in Lean operating on infinite registers.

```

1922      1 builtin.module {
1923      2   riscv_func.func @main(%0 : !riscv.reg<t1>) -> !riscv.reg {
1924      3     %1 = riscv.li 8 : !riscv.reg<t0>
1925      4     %2 = riscv.li 31 : !riscv.reg<t2>
1926      5     %3 = riscv.sra %0, %2 : (!riscv.reg<t1>, !riscv.reg<t2>) -> !riscv.reg<t1>
1927      6     %4 = riscv.and %3, %1 : (!riscv.reg<t1>, !riscv.reg<t0>) -> !riscv.reg<t0>
1928      7     %5 = riscv.add %4, %3 : (!riscv.reg<t0>, !riscv.reg<t1>) -> !riscv.reg<t0>
1929      8     riscv_func.return %5 : !riscv.reg<t0>
1930      9   }
1931     10 }

```

(b) Post-register allocation: RISC-V assembly IR with register assigned as part of the type after running the enhanced XDSL register allocator. `t0` and `t1` refer to the caller-saved temporary RISC-V register according to the RISC-V calling conventions.

```

1932      1 .text
1933      2 main:
1934      3   li t0, 8
1935      4   li t2, 31
1936      5   sra t1, t1, t2
1937      6   and t0, t1, t0
1938      7   add t0, t0, t1
1939      8   ret

```

(c) Post-register allocation: Producing RISC-V assembly using the assembly printer given the register allocated IR in b.)

Fig. 81. Converting SSA-based RISC-V assembly into register-allocated machine code for the 64-bit RISC-V target.

- `riscv`: Parses a program in the RISC-V ISA IR into Lean-internal data structures, analogous to what `verbose` does for LLVM IR. This flag is intended to allow re-parsing the output of the instruction selector back into Lean types for further processing.
- `passriscv64`: Invokes the instruction selector with all optimizations disabled, including pseudo-instruction lowering; only the canonical DCE and CSE passes are applied. The `.mlir` file is parsed into a Lean-internal data structure, lowered by the verified ISel multi-rewrite engine applying SelectionDAG patterns, and the selected program is printed to standard output. If any step fails, an error message is printed and exit code 1 is returned.
- `passriscv64_optimized`: Analogous to `passriscv64`, but with the verified GlobalISel optimizations and pseudo-instruction lowering enabled.

With `opt` in Lean, we mirror realistic compiler tooling within an interactive theorem prover and integrate our verified instruction selector into a command-line compilation flow. The selector is further embedded into an end-to-end LLVM IR-to-RISC-V backend via MLIR's reimplement in Python.

10 Evaluation

We assess whether our goal of verified instruction selection for a subset of LLVM IR using official ISA semantics with bit-blasting support is achieved by answering the following research questions:

RQ1: Does our `riscv` dialect cover the core of the RISC-V ISA, match RISC-V’s authoritative semantics, and support automated instruction selection verification via verified bit-blasting solvers? (Section 10.1)

RQ2: Can we leverage certified instruction selection patterns to build a verified instruction selector, and how does it compare to LLVM’s two main instruction selectors in terms of code size? (Section 10.2)

RQ3: What effort is required to scale the verified selector with new certified LLVM rewrites that can be ported into a LLVM selector and can our infrastructure certify correctness in LLVM’s trusted directories, e.g., the test suite, as well as flag previous unsound LLVM `Isel` patterns? (Section 10.3)

Our results show: **(RQ1)** The `riscv` dialect provides comprehensive coverage of the RISC-V ISA instructions (91% of the base ISA, 100% of the M, and 87% of the B extension). All of its ISA instruction semantics are authoritative, where 69% can be fully bit-blasted and verified by `bv_`-`decide`. **(RQ2)** Over up to 1514 synthesized test inputs ranging across size 2-6 over the LLVM IR arithmetic fragment, our verified selector matches with up to 78% similarity with LLVM’s `GlobalIsel` instruction count and 48% with `SelectionDAG`’s. While the remaining differences show that `SelectionDAG` is currently most effective, our verified instruction selector creates output similar in size to `GlobalIsel`. **(RQ3)** Scaling our verified selector with new patterns within the fragment supported by `bv_``decide` requires only mechanizing the rewrite in MLIR syntax, where the proof is then discharged automatically. With this approach, previously unsound LLVM `Isel` patterns are flagged as soon as they are expressed as rewrite rules, and legacy `SelectionDAG` combiners, as well as LLVM’s code generation test, can likewise be mechanized and certified. Hence, our infrastructure strengthens trust in LLVM’s testing directories while exposing minimal proof effort.

10.1 Authoritative RISC-V ISA Dialect For Compiler Verification

To demonstrate the coverage of `riscv`, we quantify the RISC-V ISA supported by it and indicate for which ISA instructions `riscv` enables automated ISA verification via bit-blastable semantics supported by the verified bit-blasting solver `bv_``decide` (Figure 82).

ISA Subset	Function	# Instructions	LLVM	Sail	riscv	Bitblastable
RV64I	Base ISA	33	✓	✓	91 %	100 %
M	Mul/Div	13	✓	✓	100 %	85 %
B	Bitmanipulation	41	✓	✓	87 %	66 %
Pseudo-Ops	Aliasing	24	✓	-	46 %	100 %
Total		111			86%	69%

Fig. 82. Our Lean `riscv` dialect supports 86% of RISC-V’s core integer instructions (base ISA, mul/div and bit-manipulation extensions) only omitting memory operations, enabling for 69% scalable proof automation via verified bitblasting of the supported ISA instructions. All instructions modeled in `riscv` are targeted by LLVM’s RISC-V backend, and all, except pseudo-instructions, are formalized in the Sail RISC-V processor model.

`riscv` supports 30 out of 33 base ISA instructions, hence providing 91% coverage over the instructions suitable for implementing a simple load/store computer. Namely, we support all base

instructions except for memory operations (load, store, and fence), which cannot be modeled as pure operations, where source and destination registers directly correspond to operands and results.

`riscv` is designed declaratively (Section 4.1), contrasting backend dialects such as found in MLIR or xDSL, where the semantics of an operation are defined via the mapping performed by the compiler. Each `riscv` operation is assigned to a denotation based on the instructions' authoritative semantics as defined in the Sail-to-Lean translation of the ratified Sail RISC-V processor model. Hence, `riscv` not only allows for using an assembly IR in Lean, but also directly allows for reasoning over such using authoritative ISA semantics. While ISA reasoning in ITPs has previously been enabled, e.g., by Sail-to-Lean, the definitions based on the official RISC-V model rely on conversions between bitvectors, integers, and natural numbers and are tuned for the needs of the processor, instead of compiler verification. This poses an obstacle for using such mechanizations in compiler verification via bit-blasting solvers, which is also one of the reasons why verification projects, e.g., [3], resort to hand-written ISA semantics.

`riscv` improves this state of the art by supporting authoritative ISA semantics which are provably converted into the bitvector domain (Figure 83a) and therefore integrable with Lean's verified bit-blaster, enabling proof automation. The result is a minimized, semantically faithful model of the RISC-V 64-bit base ISA (Figure 83a) as well as the M (Figure 83c) and B (Figure 83b) extensions, which is 69% fully bit-blastable and amenable for automatic ISA verification.

Instruction	Sail	Sail-2-Lean	Lean Pure		RISC-V Dialect
			Nat	BitVec	
lui	✓	✓	-	✓	✓
auipc	✓	✓	-	✓	✓
addi	✓	✓	-	✓	✓
andi	✓	✓	-	✓	✓
ori	✓	✓	-	✓	✓
xori	✓	✓	-	✓	✓
addiw	✓	✓	-	✓	✓
add	✓	✓	-	✓	✓
slli	✓	✓	✓	✓	✓
sub	✓	✓	-	✓	✓
and	✓	✓	-	✓	✓
or	✓	✓	-	✓	✓
xor	✓	✓	-	✓	✓
sll	✓	✓	✓	✓	✓
srl	✓	✓	-	✓	✓
sra	✓	✓	✓	✓	✓
addw	✓	✓	-	✓	✓
subw	✓	✓	-	✓	✓
sllw	✓	✓	✓	✓	✓
srlw	✓	✓	-	✓	✓
sraw	✓	✓	✓	✓	✓
slti	✓	✓	-	✓	✓
sltiu	✓	✓	-	✓	✓
srli	✓	✓	✓	✓	✓
srai	✓	✓	✓	✓	✓
slliw	✓	✓	✓	✓	✓
srliw	✓	✓	✓	✓	✓
sraiw	✓	✓	-	✓	✓
slt	✓	✓	-	✓	✓
sltu	✓	✓	-	✓	✓
load	✓	✓	-	-	-
store	✓	✓	-	-	-
fence	✓	✓	-	-	-

(a) RISC-V Base ISA

Instruction	Sail	Sail-2-Lean	Lean Pure		RISC-V Dialect
			Nat	BitVec	
add.uw	✓	✓	-	✓	✓
sh1add.uw	✓	✓	✓	✓	✓
sh2add.uw	✓	✓	-	✓	✓
sh3add.uw	✓	✓	-	✓	✓
sh1add	✓	✓	-	✓	✓
sh2add	✓	✓	-	✓	✓
sh3add	✓	✓	-	✓	✓
slli.uw	✓	✓	-	✓	✓
rol	✓	✓	✓	✓	✓
ror	✓	✓	-	✓	✓
rolw	✓	✓	✓	-	✓
rorw	✓	✓	-	-	✓
sext.b	✓	✓	-	✓	✓
sext.h	✓	✓	-	✓	✓
zext.h	✓	✓	-	✓	✓
rori	✓	✓	✓	✓	✓
roriw	✓	✓	-	✓	✓
andn	✓	✓	-	✓	✓
orn	✓	✓	-	✓	✓
xnor	✓	✓	-	✓	✓
max	✓	✓	✓	-	✓
maxu	✓	✓	-	-	✓
min	✓	✓	✓	-	✓
minu	✓	✓	✓	-	✓
bclr	✓	✓	✓	✓	✓
bclri	✓	✓	✓	✓	✓
bext	✓	✓	✓	✓	✓
bexti	✓	✓	✓	✓	✓
binvi	✓	✓	✓	✓	✓
bset	✓	✓	✓	✓	✓
bseti	✓	✓	✓	✓	✓
czero.eqz	✓	✓	-	✓	✓
czero.nez	✓	✓	-	✓	✓
orc.b	✓	✓	-	-	-
rev8	✓	✓	-	-	-
clz	✓	✓	-	-	-
clzw	✓	✓	-	-	-
ctz	✓	✓	-	-	-
ctzw	✓	✓	-	-	-
cpop	✓	✓	-	-	-
cpopw	✓	✓	-	-	-

(b) RISC-V B Extension

Instruction	Sail	Sail-2-Lean	Lean Pure		RISC-V Dialect
			Nat	BitVec	
mul	✓	✓	✓	✓	✓
mulw	✓	✓	✓	✓	✓
mulh	✓	✓	✓	✓	✓
mulhu	✓	✓	✓	✓	✓
mulhsu	✓	✓	✓	✓	✓
divw	✓	✓	✓	-	✓
divuw	✓	✓	✓	-	✓
div	✓	✓	✓	✓	✓
divu	✓	✓	✓	✓	✓
remw	✓	✓	✓	✓	✓
rem	✓	✓	✓	✓	✓
remuw	✓	✓	✓	✓	✓
remu	✓	✓	✓	✓	✓

(c) RISC-V M Extension

Instruction	Sail	Sail-2-Lean	Lean Pure		RISC-V Dialect
			Nat	BitVec	
li	-	-	-	✓	✓
mv	-	-	-	✓	✓
not	-	-	✓	✓	✓
neg	-	-	-	✓	✓
negw	-	-	-	✓	✓
sext.w	-	-	✓	✓	✓
zext.b	-	-	-	✓	✓
seqz	-	-	-	✓	✓
snez	-	-	✓	✓	✓
sltz	-	-	-	✓	✓
sgtz	-	-	-	✓	✓

(d) RISC-V Pseudo-instructions

Fig. 83. Our riscv dialect covers 86% of the RISC-V base ISA, B and M extensions and provides support for pseudo-instructions. The majority of operations offer side-effect-free pure semantics at the bitvector level, which enables automatic verification via bitblasting.

The RISC-V M extension is fully covered using authoritative semantics and has 85% bit-blastable semantics. Similarly, the B extensions (Figure 83b) are supported with 87%, where all subextensions, except for the ZBB extension with 67%, are fully implemented. ZBB does not yet have complete coverage in `riscv`, as population-count instructions are not supported by the `bv_decide` bit-blaster yet.

The semantics of `riscv` are practical for compiler verification because reasoning about pure operations can be automated using `bv_decide`, and avoids large functions found in mechanizations extracted from Sail¹⁰, but still allows for verification against official semantics. In addition, `riscv` mechanizes pseudo-instructions (Figure 83d). This supports a step towards trustworthy foundations in compiler reasoning as pseudo-operations are not contained in processor formalizations, e.g., Sail, though relevant for validating backends. Popular compilers, e.g., LLVM omit them in their textual assembly output as indicated in Figure 82. Moreover, our evaluation (Figure 82) indicates that all instructions modeled in `riscv` are targeted by LLVM and formalized in Sail, with the exception of the pseudo-operations only found in LLVM-based compilers. `riscv` covers 11 pseudo-operations, thereby focusing on those that alias with base ISA, M, or B instructions, leaving memory operation aliases for future work. As we alias the pseudo-instruction semantics with native ISA instructions, they also have full bit-blasting support. Hence, we not only propose a mechanization of a set of RISC-V pseudo-operations but provide semantics shaped for practical proof automation. Future work can extend support to load/store, floating-point, and control-flow operations to approach 100% coverage of the user-space ISA.

Overall, our `riscv` dialect modeling the RISC-V 64-bit ISA covers 86% of the RISC-V base, B, and M instructions, including pseudo-instructions, which, to our knowledge, had not been mechanized before. The semantic denotation of all `riscv` instructions is provably extracted from the authoritative, mechanized RISC-V ISA specification and 69% of our semantics are formalized in a bit-blasting-friendly format. This allows for automatic proof discharge when reasoning at the RISC-V ISA level.

10.2 Certified Instruction Selection Against LLVM

To assess how competitive our verified instruction selection, which leverages certified SelectionDAG and GlobalISel rewrites, is against LLVM’s existing RISC-V instruction selection flows, we evaluate the instruction count of the selected output across different programs¹¹.

Set Up. We obtain input LLVM IR programs from `mlir-fuzz`¹²[13] and synthesize programs over the arithmetic fragment of LLVM IR, hence floating points, memory operations, and control flow are out of scope. We evaluate our selector on single-basic-block programs of varying sizes, comparing against LLVM, which performs analyses, such as known-bits analysis and SelectionDAG combinators, which operate across instructions within the block. The exact two compilation flows whose output we compare (Section 9) are (1) our prototype backend using our certified instruction selector at its core, (2) a compilation flow based on MLIR/LLVM infrastructure invoking the LLVM RISC-V backend at its default optimization level, lowering the same input as our prototype backend. The verified instruction selector is invoked with `passriscv64_optimized`, enabling verified GlobalISel rewrites and pseudo-instruction lowering, and LLVM at `llc`’s default optimization level of `-O2`, once relying on the legacy SelectionDAG selector and once enabling the modern GlobalISel framework.

¹⁰<https://github.com/opencompl/sail-riscv-lean>

¹¹We use LLVM from <https://github.com/llvm/llvm-project/> at version 21.1.0 and commit hash 5abec20b25be01866b1c6f75f6f45426d2b4f87b

¹²MLIR fuzz is a fuzzer for MLIR that can exhaustively generate all programs of a given size and MLIR dialect. We extend MLIR fuzz with the arithmetic fragment of the LLVM IR.

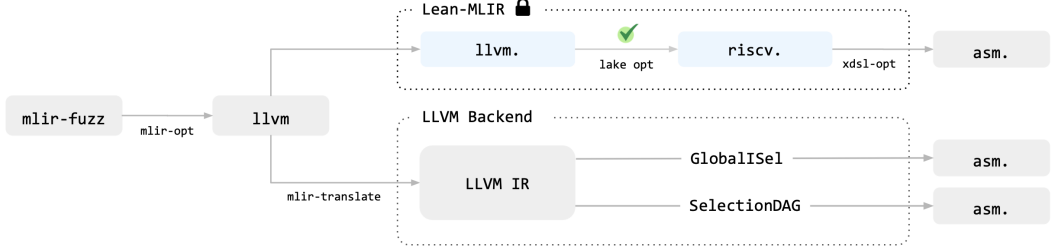


Fig. 84. We compare our verified instruction selector with the two LLVM instruction selectors: SelectionDAG and GlobalISel.

Result. Over several randomly generated sets of up to 1514 program inputs drawn from the arithmetic fragment including the [IR](#) instructions `shl`, `lshr`, `ashr`, and, `or`, `xor`, `add`, `sub`, `mul`, `trunc`, `zext`, `sxt`, `icmp`, `select` and the instruction attribute flags `exact`, `disjoint`, `nsw`, `nuw`, our verified selector successfully handles all cases (100% selection rate) for the `i64` type and achieves instruction counts comparable to GlobalISel. The smaller the program, as indicated by its instruction count, the more often the three selectors align. For programs of size 2, the verified selector aligns with GlobalISel, producing an equal instruction count in 78% of the cases. The remaining discrepancies arise from missing optimizations or differences in selection order, which lead to distinct optimization opportunities. However, SelectionDAG outperforms both (Figure 88). As program size increases, SelectionDAG continues to exhibit an optimized selection compared to both, GlobalISel and our verified selector, owing to the mature optimizations it incorporates, e.g., with an input program size of 4 in Figure 89. Figure 85 and Figure 86 illustrate concrete cases in which SelectionDAG ISel achieves heavier optimized ISel than both LLVM’s GlobalISel and the verified selector.

```

1  div a1, a0, a0
2  or  a0, a1, a0
  
```

(a) GlobalISel’s and our verified instruction selection.

```

1
2  ori a0, a0, 1
  
```

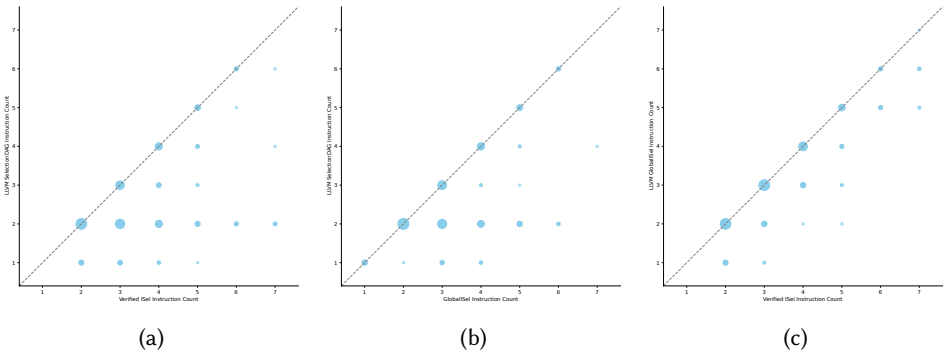
(b) SelectionDAG instruction selection

Fig. 85. A program of size 2 in which SelectionDAG applies optimizing combines that are not yet supported in either our verified selector or GlobalISel. Both GlobalISel and our selector produce 2 instructions, whereas SelectionDAG combines them into a single `ori`.

Figure 87 illustrates the divergence between the selectors and emphasizes that compared to the unverified, modern GlobalISel selector, the verified instruction selection diverges both less frequently and less pronounced, as indicated by the approximate diagonal (Figure 87c). In contrast, both GlobalISel as well as the verified selector diverge from SelectionDAG (Figure 87a & Figure 87b). The similarity between GlobalISel and our verified selector arises from the fact that our selector also leverages GlobalISel’s optimizing combines, leading both to perform similar optimizations. The divergences that occur can be attributed to missing dataflow information in our current selector.

2206 1 rem a1, a0, a0
2207 2 sra a2, a0, a1
2208 3 xor a1, a1, a2
2209 4 sra a0, a0, a1
2210 (a) GlobalSel and our verified instruction selection
2211 yield a program count of 4.
2212 1
2213 2
2214 3
2215 4 sra a0, a0, a0
2216 (b) SelectionDAG instruction selection produces a single
2217 instruction for an input of size 4, thereby leveraging
2218 highly optimized selection rules.

2219 Fig. 86. SelectionDAG outperforms both GlobalSel and our verified selector. In this case, GlobalSel and the
2220 verified selector map IR instructions one-to-one onto assembly instructions, whereas SelectionDAG optimizes
2221 across a basic block and benefits from mature optimizations due to its legacy status, yielding a single selected
2222 instruction as opposed to 4 with GlobalSel and the verified selector.



2236 Fig. 87. Across 400 functions of up to size 6, the verified selector exhibits the same trend as GlobalSel which
2237 is reflected by the diagonal and the larger dots along it, indicating frequent matches in instruction count.
2238 However, as shown in (a) and (b), both selectors diverge from SelectionDAG.

2240 Figure 90 indicates the instruction count evaluation over programs up to size 6, where again both
2241 GlobalSel and our verified selector have similar trends, while SelectionDAG exhibits an optimized
2242 selection.

2243 Overall, our results demonstrate that within a verified compilation framework, it is possible
2244 to achieve realistic instruction selection that matches LLVM’s GlobalSel, the modern instruction
2245 selection framework designed to be more modular and less monolithic than its predecessor. Our
2246 experiments show that while the legacy SelectionDAG selector outperforms both GlobalSel and our
2247 verified selector with increasing program size, due to its mature and highly optimized combiners
2248 library, the LLVM community’s deliberate move toward GlobalSel reflects a trade-off between
2249 optimization strength and modularity. Crucially, our verified selector already achieves parity with
2250 this modern baseline across runs. This provides evidence that instruction selection grounded in
2251 verified rewrites can approximate the behavior of practical selectors, while preserving formal
2252 guarantees over the IR-to-ISA mapping. Despite missing optimizations, e.g., such that are enabled
2253 by a prior dataflow analysis as in SelectionDAG, the proximity of our results to LLVM’s new selector

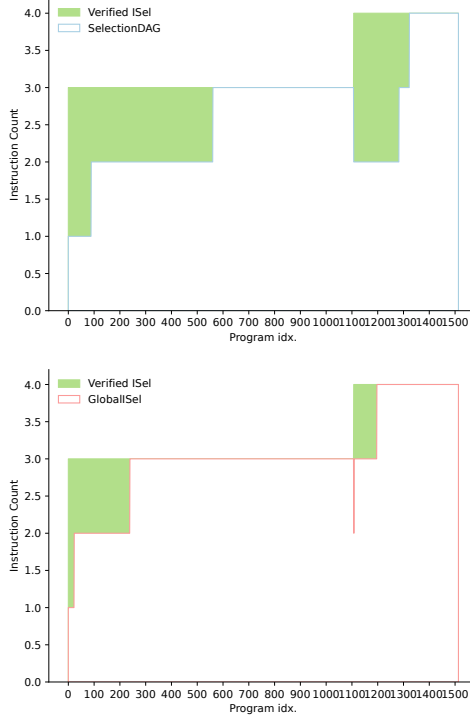


Fig. 88. We synthesized 1,514 programs over the LLVM IR arithmetic fragment of exact size 2 and compared instruction counts across selectors. The verified selector is outperformed by the legacy, heavily optimized SelectionDAG, they yield identical program sizes in 42% of the cases. However, the verified selector matches GlobalISel with 79% similarity while providing full verification. The green area illustrates the 'verification overhead' and shows the difference between the LLVM selectors' and the verified selector's instruction counts.

gives us confidence that verified instruction selection based on verified selection rules is a viable and promising path toward building trustworthy compiler backends.

10.3 Application In Realistic Compilers

We demonstrate the minimal effort required to scale the verified selector by certifying SelectionDAG combines, we propose to port it into our selector and GlobalISel. Using the same certification approach, we enhance trust in LLVM's test suite by mechanizing test cases as rewrites and demonstrate how reported, unsound LLVM ISel rules are automatically flagged, which would have prevented them from reaching production.

10.4 LLVM'S RISC-V Test Suite

As of version 21.1.0, LLVM includes test cases targeting LLVM IR to RISC-V code generation. These test cases are hand-written by individual members, and approved through community review. A code generation test case captures the lowering from LLVM IR to the ISA, which we model as a single rewrite. We encode each test case using our Lean structure (Section 5) and rely on our proof automation tactics to certify its logic against the authoritative RISC-V ISA semantics. Instead of relying on trust-based test cases, we produce mechanized proofs that certify them.

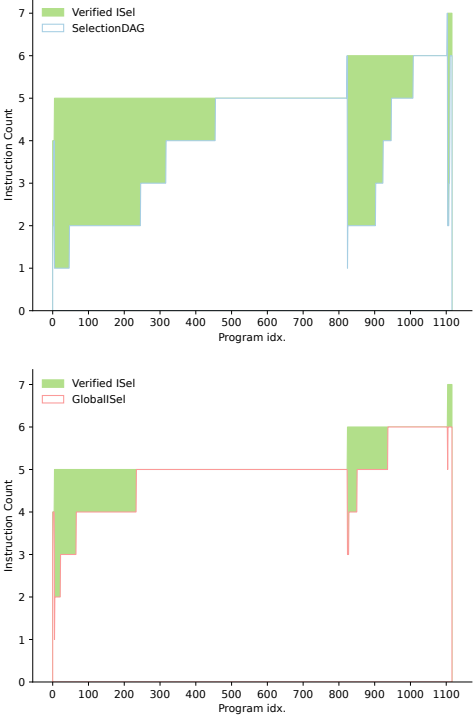


Fig. 89. We synthesized 1100 programs over the LLVM IR arithmetic fragment with the exact IR program size of 4 and compared the instruction counts across selectors. The verified selector aligns with GlobalISel instruction count with 70% matching instruction count while SelectionDAG outperforms the verified selector with the exact same instruction count in 42% .

A typical test case in the LLVM test suite targeting RISC-V code generation:

```
1 ; RUN: llc -mtriple=riscv64 -verify-machineinstrs < %s \
2 ; RUN: | FileCheck %s -check-prefix=RV64I
3
4 define i64 @add_hi_one_lo_negone(i64 %0) {
5 ; RV64I-LABEL: add_hi_one_lo_negone:
6 ; RV64I: # %bb.0:
7 ; RV64I-NEXT: li a1, -1
8 ; RV64I-NEXT: srli a1, a1, 31
9 ; RV64I-NEXT: add a0, a0, a1
10 ; RV64I-NEXT: ret
11 %2 = add nsw i64 %0, 8589934591
12 ret i64 %2
13 }
```

Fig. 91. LLVM code generation test case for the RISC-V target. The test checks that a LLVM IR add nsw with the constant 8589934591 is lowered onto a RISC-V assembly sequence.

The test case (Figure 91) checks whether the lowering of the LLVM IR program containing a signed-overflow flag add with the constant 8589934591 is lowered to the specified ISA sequence. The first line in the test specifies the static compiler llc to be invoked to compile the current file

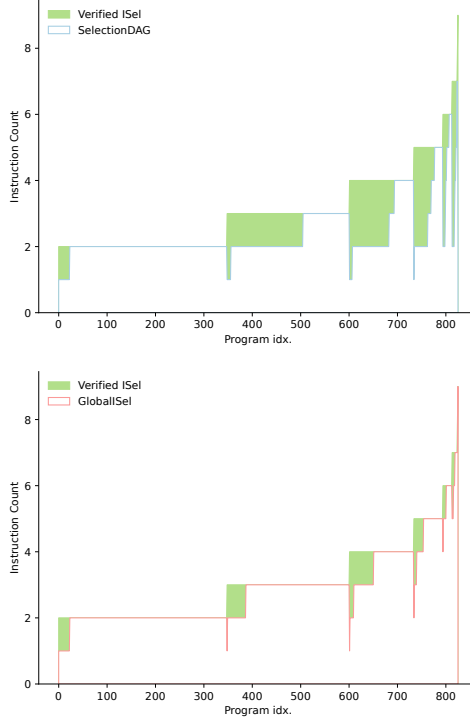


Fig. 90. We synthesized 825 programs over the LLVM IR arithmetic fragment of size up to 6 and compared instruction counts across selectors. The smaller the programs, the closer the selectors align. LLVM’s GlobalISel and our verified selector exhibit a similar trend in instruction count. SelectionDAG, however, can produce fewer instructions, and the verified selector diverges from its counts more often.

to machine code for a RISC-V 64-bit processor. The `-verify-machineinstrs` indicates to verify the machine instruction generated by `llc`. Lastly, the output is piped into the `FileCheck` [17] tool, which verifies that the output corresponds to the expected patterns annotated with `RV64I` in the same file.

Note that the test covers not only instruction selection but the work performed by the RISC-V backend. Nevertheless, we can verify the test by modeling it as a single, local rewrite rule in Lean (Figure 92).

Results We automatically prove refinement between LLVM IR and RISC-V target code for all 90 test cases in the validated suite (Figure 93) using our automation tactics `simp_lowering`, followed by a call to `bv_decide`. If our proof automation tactics neither produce a counterexample nor encounter a case outside our supported language fragment, the proof is automatically discharged. More generally, LLVM code generation can be modeled as rewrites in Lean, enabling future automation, e.g., automatic encoding, to verify a broad subset of RISC-V code generation tests. This demonstrates that our approach to verified instruction selection also supports verification of LLVM rewrites that are used to test LLVM itself.

```

2402 1 def add_hi_one_lo_negone_llvm_i64 := [LV] {
2403 2   ^entry (%arg: i64):
2404 3   %8589934591 = llvm.mlir.constant (8589934591) : i64
2405 4   %1 = llvm.add %arg, %8589934591 overflow<ns> : i64
2406 5   llvm.return %1 : i64
2407 6 }}
2408 7
2409 8 def add_hi_one_lo_negone_riscv_i64 :=
2410 9 [LV] {
2411 10  ^entry (%arg0: i64):
2412 11  %a = "builtin.unrealized_conversion_cast" (%arg0) : (i64) -> (!riscv.reg)
2413 12  %0 = li (0) : !riscv.reg
2414 13  %1 = li (1) : !riscv.reg
2415 14  %2 = sub %0, %1 : !riscv.reg
2416 15  %3 = srli %2, 31 : !riscv.reg
2417 16  %4 = add %3, %a : !riscv.reg
2418 17  %5 = "builtin.unrealized_conversion_cast" (%4) : (!riscv.reg) -> (i64)
2419 18  llvm.return %5 : i64
2420 19 }}
2421 20
2422 21 def add_hi_one_lo_negone_i64__test : LLVMPeepholeRewriteRefine 64 [Ty.llvm (.bitvec 64)] where
2423 22 lhs := add_hi_one_lo_negone_llvm_i64
2424 23 rhs := add_hi_one_lo_negone_riscv_i64
2425 24 correct:=... PROOF AUTOMATICALLY DISCHARGED

```

Fig. 92. Certifying a test case using our instruction selection approach using MLIR and SSA assembly syntax. The codegen test is implemented as a lowering pattern and checked using our proof automation. If the proof succeeds, the pattern is certified; otherwise, `bv_decide` produces a counterexample indicating unsoundness.

LLVM's RISC-V Test File	Certified Test Cases	Test File Coverage
add-imm	11/13 (memory)	84%
alu 16	19/22 (intrinsics)	86%
add-imm64-to-sub	5/ 5	100 %
add-sext-shl-constant	6/12 (memory)	50%
alu 32	24/24	100 %
alu 64	25/35 (intrinsics)	72%

Fig. 93. We verified LLVM's trusted code generation test files by abstracting them as peephole optimizations and implementing them as patterns in our certified pattern library. For each file, we verified all tests within our LLVM IR fragment, when full verification was not possible, we specify the verified part and the remaining instruction classes required but not yet supported, e.g., LLVM IR intrinsics.

10.5 Reproducing Known Bugs In The LLVM Instruction Selector

We show how previously reported miscompilation patterns in LLVM's instruction selection, e.g., those disclosed in June 2025, would have been flagged before reaching production. First, automatically verifying them as part of our pattern library alerts developers to the unsound rewrite while imposing minimal proof burden, making it practical for trustworthy future production rewrites. We apply the approach to recent bugs in LLVM's RISC-V backend as listed in a LLVM bug report¹³ [3] and pick such that fall into our language fragment.

One unsound pattern implemented in SelectionDAG specifies that when a signed division is followed by a signed remainder operation with the same operands in the same basic block, the remainder should be replaced with a computation of dividend minus the multiplication of the quotient and divisor (Figure 94).

¹³<https://github.com/regehr/alive2/blob/arm-tv/BugList.md>

```

2451 1      if (SDNode *RemNode = DAG.getNodeIfExists(ISD::SREM, N-&gtgetVTList(),
2452 2          { N0, N1 }))) {
2453 3          SDValue Mul = DAG.getNode(ISD::MUL, DL, VT, V, N1);
2454 4          SDValue Sub = DAG.getNode(ISD::SUB, DL, VT, N0, Mul);
2455 5          CombineTo(RemNode, Sub);

```

Fig. 94. A srem B gets replaced by A - C * B where C is the result of the signed division. The entire remainder operation gets replaced with the result of the subtraction.

However, the rewrite is not valid when the exact flag is set for the signed division. In that case, if the division is not exact, poison will be returned and poison taints subtraction and multiplication, therefore replacing the result of the signed remainder with poison. However, the original pattern returns a concrete value since both of its operands are concrete.

Blindly mechanizing the pattern in our Lean framework makes the proof automation fail, and the unsound pattern requires the goal state in Figure 96c to be proven, which is impossible. A concrete value is never refined by poison, which is made explicit in the proof obligation. Moreover, while GlobalISel aims to be the new, improved selector, incorrect patterns are being deployed, as a bug revealed in April 2025¹⁴. During selection of the arithmetic fragment of LLVM IR, a select on a sub instruction and a sub instruction with the nsw flag was incorrectly combined during ISel into a select of the same sub with nsw, which is not a refinement¹⁵. When mechanizing the rewrite logic in Lean, our proof automation fails and requires a goal state of false to be proven. Hence,

<pre> 2472 1 def llvm_select_bug_lhs := [LV] { 2473 2 ^entry (%x: i64, %y: i64, %c : i1): 2474 3 %1 = llvm.sub %x, %y : i64 2475 4 %2 = llvm.sub %x, %y overflow<nsw> : i64 2476 5 %3 = llvm.select %c, %2, %1 : i64 2477 6 llvm.return %3 : i64 2478 7 } </pre> (a)	<pre> 1 def llvm_select_bug_rhs := [LV] { 2 ^entry (%x: i64, %y: i64, %c : i1): 3 %2 = llvm.sub %x, %y overflow<nsw> : i64 4 %3 = llvm.select %c, %2, %2 : i64 5 6 llvm.return %3 : i64 7 } </pre> (b) .
--	--

Fig. 95. Wrong match-and-replace pattern in LLVM’s GlobalISel selector. The nsw flag should not propagate with this combining pattern, since a sub with the flag (b) can yield poison while a sole subtraction doesn’t (a).

we can prevent developers from registering patterns in our selector or LLVM, which were bugs in LLVM until mid-2025. Notice, using our framework only required implementing the pattern in Lean using MLIR syntax, with no proof obligations placed on the user, still strong enough to prevent bugs present in today’s compilers.

10.6 Scaling ISel With New Rewrites

The verified instruction selector scales by adding new rewrites to lowering passes listed in Section 8.2. In addition, connecting to LLVM today, LLVM backend developers can use our verification infrastructure to check candidate rewrites, relying on our tactics to certify patterns. A proposed workflow is to certify a potential rewrite and then confidently deploy it, e.g., either into GlobalISel or our verified selector, by registering it in the corresponding list initializer. Figure 97b shows the mechanization of one of the four rewrites from LLVM’s RISC-V SelectionDAG¹⁶, which we

¹⁴The corresponding bug report: <https://github.com/llvm/llvm-project/issues/145360>

¹⁵sub yields always a non-poison value while subn nsw can be poison, hence replacing the operands in the select with the possible poison instruction is not a refinement [17].

¹⁶The LLVM SelectionDAG combines are taken from: <https://github.com/llvm/llvm-project/blob/main/llvm/lib/CodeGen/SelectionDAG/DAGCombiner.cpp>

```

25001 ^entry (%a: i64, %b: i64):
25012   %0 = llvm.sdiv exact %a, %b : i64
25013   %1 = llvm.srem %a, %b : i64
25024
25025   llvm.return %1 : i64
2503

```

(a) Original instruction sequence formalized in Lean. First signed division between the two i64 operands is computed, followed by a subsequent signed remainder.

```

25081 ⊢ PoisonOr.value (x.srem x1) ⊆ PoisonOr.poison
2509

```

(c) Goal state indicating that the pattern is wrong, since a concrete value is never refined by poison, as poison is a UB form.

Fig. 96. The combining pattern in Figure 94 encodes that whenever a signed division is encountered, a signed remainder using the same operands can be rewritten into a corresponding multiplication and subtraction. However, this optimization pattern is not a refinement that returns poison in scenarios where the original pattern returned a concrete value.

identified as not supported in GlobalISel nor in our verified selector, but can now be ported with correctness guarantees. Hence, our verified instruction selector not only provides correctness guarantees but also serves as a framework for validating future LLVM ISel rewrites.

```

25221 // fold (add x, shl(0 - y, n)) -> sub(x, shl(y, n))
2523

```

(a) Logic behind SelectionDAG combiner applied during instruction selection, not yet ported into GlobalISel or our verified selector.

```

25261 def fold_add_sub_mask_selectiondag_lhs := [LV] {
25272   ^entry (%y: i64, %n: i64, %x: i64):
25273     %c = llvm.mlir.constant (0) : i64
25284     %0 = llvm.sub %c, %y : i64
25295     %1 = llvm.shl %0, %n : i64
25306     %2 = llvm.add %x, %1 : i64
25307     llvm.return %2 : i64
25318 }
25329 def fold_add_sub_mask_selectiondag_rhs := [LV] {
253310   ^entry (%y: i64, %n: i64, %x: i64):
253311     %0 = llvm.shl %y, %n : i64
25342     %1 = llvm.sub %x, %0 : i64
25343     llvm.return %1 : i64
25354 }
25365 def fold_add_sub_mask_rewrite : LLVMPeepholeRewriteRefine 64 [Ty.llvm (.bitvec 64), Ty.llvm (.bitvec 64), Ty.llvm (.bitvec 64)] where
2537   lhs:= fold_add_sub_mask_selectiondag_lhs
2538   rhs:= fold_add_sub_mask_selectiondag_rhs
2539 SelectionDAG Combiner proof automatically discharged
2540

```

(b) Mechanization of the pattern in Lean over LLVM IR semantics ensures that the rewrite can be trustworthily ported into existing infrastructure and the verified instruction selection.

10.7 Conclusion

We presented a verified instruction selector for the LLVM IR arithmetic fragment that exhibits similar trends to LLVM's modern instruction selection framework, GlobalISel. By incorporating verified,

canonical SelectionDAG rules our selector builds on LLVM rewrites and is further complemented with a subset of certified GlobalISel optimizing rewrites. The IR-to-ISA mapping is fully verified up to semantic refinement against the official RISC-V ISA specification, and scalability is ensured through proof automation via a verified bit-blaster, enabling new patterns to be added with minimal proof effort. Our work aims to strengthen trust in LLVM by motivating developers to verify their selection rules before production, using familiar syntax in our framework, and thereby verify their ISA lowerings. At the same time, it motivates verified compilation, starting with an individual compilation pass that perform the crucial IR-to-ISA mapping. As our evaluation shows, within the arithmetic fragment, our approach would have prevented LLVM bugs persisting until June 2025. In conclusion, our verified instruction selector not only establishes a practical path toward trustworthy instruction selection for LLVM IR, but also provides a concrete step toward the larger goal of automation-supported, verified compilation.

Future Work Suggestions: This work introduced riscv as a lightweight, pure ISA model amenable to automatic compiler verification via a verified bit-blaster in Lean. Future directions include extending the ISA toward full user-space coverage and enriching the dialect with floating-point and memory operations. Beyond the verified instruction selection pass for LLVM IR’s arithmetic fragment, additional backend passes, such as legalization, register allocation and instruction scheduling, can modeled and verified, yielding a backend composed of verified compilation passes. While working with the Sail processor model, an interesting future direction is to automatically synthesize such dialects or backend passes from formal processor and ISA models. Our current selector can also be extended to handle memory and branching instructions as Lean-MLIR matures, while dataflow analyses within an instruction selector or across the backend present further opportunities for formalization and verification. Finally, we encourage the LLVM community to prevent instruction selection bugs by verifying patterns up front, e.g., using our framework, thereby reducing the risk of miscompilation and moving closer to trustworthy LLVM backends.

11 Related Work

Our work contributes to verified backend compilation, thereby focusing on trustworthy verification and rewrites of realistic compilers. In this chapter, we survey related efforts aimed at increasing trust throughout the compilation stack.

Formal IR And ISA Semantics To verify a compiler backend and thereby distinguish miscompilation from correct compilation, consensus on the specification of the input and output languages is required. In LLVM, the intended semantics of LLVM IR are officially documented in [22]; however, not exhaustive, and there historically exists a lack of coordination to close this gap. Vellvm [38] proposed a formalization of LLVM IR accompanied by a Rocq [15] framework for reasoning about LLVM IR programs. Vellvm allows for reasoning about IR-to-IR rewrites while we widen towards compiler IR to ISA lowering within our fragment. The Lean-MLIR¹⁷ [4] project implements semantics for the arithmetic fragment of LLVM IR, which we leverage in our work. Their semantics align with those derived in the Alive project [23], which iterated over underspecified parts of LLVM IR semantics and fixed a subset of discrepancies. We tighten a gap for formal RISC-V semantics by mechanizing pseudo-instructions in Lean, which are absent in formal processor models. However, the mechanization of ISA semantics exhibits mature projects like Sail [1], an architecture DSL used to formalize the ISA of ARM [35], x86 [7], and CHERI [37] processors as in [2]. Still, compiler verification projects such as Alive [23] and CompCert [20] made little use of such mechanized models and derived ISA models by hand. Contrary, we propose to leverage processor models and provably extract official semantics for verification. While [32] used processor models to verify a

¹⁷<https://github.com/opencompl/lean-mlir>

correctness property of a RISC-V-based processor with far more complexity and detail than required for compiler verification, e.g., hardware exceptions and interrupts, we extracted a subset from a processor model practical for usage in compiler verification.

Design Of Low-level IRs *ISA* dialects are widely supported in mainstream compiler frameworks like xDSL [12] and MLIR [18], which exhibit a RISC-V [12] or x86 [28] dialect. However, their operation semantics are defined via the lowering performed by the compiler.

Verified Compilation Formal methods-based compiler design has been the topic of various research projects, where we identified two prominent streams. The first implements a fully verified compilation infrastructure, where correctness is verified once and for all. The second approach verifies existing mainstream compilers using translation validation [3]. Our work is positioned between directing towards integrating end-to-end verified components, however, into the development of today’s mainstream compilers.

Fully Verified Compilation: The prominent example is CompCert [21], a compiler for the C programming language, "[...] which is formally verified, using machine-assisted mathematical proofs, to guarantee the absence of compiler bugs." [20]. CompCert compiles C programs into machine code, including RISC-V [21]. Verified in the Rocq proof assistant, CompCert relies on the strong expressiveness provided by *ITPs*, like our work does. Compilation progresses through several *IRs* and passes before outputting assembly code, also supporting various levels of abstraction of a compiler pipeline. However, enabling new, simple optimizations in fully verified compiler toolchains requires adapting the compiler core verification infrastructure, exposing a high proof burden, and being used in niche contexts. In contrast, our approach is flexible by design through independent rewrites that can be verified in isolation by the user. Moreover, CompCert’s *ISA* semantics, analog to those used in [3], are manually derived. Nevertheless, CompCert is a mature end-to-end project that provides formal guarantees for a full compilation pipeline, thereby including control flow and memory operations, and is being used in safety-critical environments.

Translation Validation: Contrasting CompCert, Alive [24] and its successor Alive2 [23], are projects that showcase verification within the mainstream compiler design space. Alive2 verifies the correctness of LLVM IR optimizations by checking input-output refinement via an SMT solver, while Alive is a DSL for writing optimizations and automatically proving them correct [24], similar to our work, where a rewrite can be formalized in familiar *IR* or *ISA* syntax for verification in Lean. However, as a translation validation tool, Alive doesn’t need the compiler to be proven correct, which makes it practical for existing production compilers, but it isn’t a verified compiler component. Alive reduces miscompilation risk, e.g., a developer can verify their rewrite before upstreaming. As it relies on an SMT solver, it trusts the large unverified codebase of Z3 [8]. Similar to Alive, which has a high degree of push-button automation, our work can automatically prove rewrites.

Compiler Backend Verification Verification focusing on the correctness of a single back-end pass, such as instruction selection hasn’t been performed for LLVM. Instead, Regehr et Al. [3] used translation validation tools to validate input and output of the AArch64 (64-bit ARM) LLVM backend by lifting assembly into LLVM IR to subsequently reuse Alive. While ARM-tv [3] directs attention toward compiler backends and their sensitivity to bugs, it does not offer formal correctness guarantees, unlike CompCert or our instruction selector. ARM-tv checks rewrites and code for refinement, using an SMT solver and, moreover, an unverified ARM-to-LLVM-IR lifter, thereby requiring trust in these unverified translations before validating the backends work. While undoubtedly increasing the trust in the LLVM ARM backend and providing high feature coverage, e.g. memory and floating point operations, as well as desirable support for automation, the work is built upon a fragile trust chain, such as manually derived *ISA* semantics. The *Instruction Selection* (*ISel*) component itself is subject to works such as [5], where formal rules were used to automatically

synthesize selectors. They offer a strong degree of automation; however, they are not connected to day-to-day mainstream compilers like LLVM.

Certified Optimizing Compiler Rewrites Various efforts verified local rewrite patterns, e.g., Peek [27], a framework for expressing, verifying, and executing machine code transformations for x86 within CompCert [21]. Similar to our work, they certify the patterns and allow their execution directly over programs. However, Peek requires the rewrites to be linear and adjacent. Contrary to the verified rewriter in Lean-MLIR [4], which can optimize across non-linear instruction sequences. Peek explicitly models side effects and includes memory-related rewrites, and supports that applying locally semantics-preserving rewrites preserves the global meaning of the program. Another project on verifying a library of rewriting patterns is [4], which verified peephole rewrites in LLVM’s InstCombine pass, or by Alive [24] and AliveInLean [19]. However, they focus on the stack around LLVM IR, no low-level backend rewrites, nor the error-prone instruction combines of LLVM’s instruction selector. Given the cumulative power of local optimizing rewrites, various verification efforts around them exist, e.g., [30], developed verification tooling for rewrites in Halide. Verification projects often reason over equality between the find- and replace- rewrite pattern instead of reasoning for correctness via refinement, which is the correctness theorem of a compiler supported in [3, 21] as well as in our work.

Beyond Formal Methods-Based Compiler Validation Besides formal verification, which proves the absence of bugs, as demonstrated in [21], testing, e.g., differential testing, and fuzzing tools have been highly effective in identifying compiler bugs. Several defects in optimizers and lowering passes of commercial compilers, including LLVM, have been discovered by tools such as those described in [17, 34]. Recent approaches combined translation validation and testing approaches [10] and continue to find miscompilation defects. After a decade, it is worth rethinking why such tools continue to find bugs in LLVM rewrites and whether it is time for a fundamental design shift towards fully verified development with verified single passes as a first step.

12 Conclusion

References

- [1] Alasdair Armstrong. [n. d.]. The Sail ISA specification language. <https://github.com/rem-s-project/sail>. Accessed: 2025-06-16.
- [2] Alasdair Armstrong, Thomas Bauereiss, Brian Campbell, Alastair Reid, Kathryn E. Gray, Robert M. Norton, Prashanth Mundkur, Mark Wassell, Jon French, Christopher Pulte, Shaked Flur, Ian Stark, Neel Krishnaswami, and Peter Sewell. 2019. ISA semantics for ARMv8-a, RISC-V, and CHERI-MIPS. *Proc. ACM Program. Lang.* 3, POPL, Article 71 (Jan. 2019), 31 pages. doi:10.1145/3290384
- [3] Ryan Berger, Briles, Mitch Nader Boushehrinejad Moradi, Nicholas Coughlin, Kait Lam, Nuno P. Lopes, Stefan Mada, Tanmay Tirpankar, and John Regehr. 2025. Translation Validation for LLVM’s AArch64 Backend. In *OOPSLA ’25: Proceedings of the ACM international conference on Object oriented programming systems languages and applications*. In press, Accessed: 2025-08-23.
- [4] Siddharth Bhat, Alex Keizer, Chris Hughes, Andrés Goens, and Tobias Grosser. 2024. Verifying Peephole Rewriting in SSA Compiler IRs. In *15th International Conference on Interactive Theorem Proving (ITP 2024) (Leibniz International Proceedings in Informatics (LIPIcs), Vol. 309)*, Yves Bertot, Temur Kutsia, and Michael Norrish (Eds.). Schloss Dagstuhl – Leibniz-Zentrum für Informatik, Dagstuhl, Germany, 9:1–9:20. doi:10.4230/LIPIcs.ITP.2024.9
- [5] Sebastian Buchwald, Andreas Fried, and Sebastian Hack. 2018. Synthesizing an instruction selection rule library from semantic specification. In *CGO ’18: Proceedings of the 2018 International Symposium on Code Generation and Optimization*. ACM, 300 – 313. <https://doi.org/10.1145/3168821>
- [6] Q. Colombet and K. Beyls. 2025. *LLVM Code Generation: A Deep Dive Into Compiler Backend Development*. Packt Publishing, Limited.
- [7] L.B. Das. 2010. *The X86 Microprocessors: Architecture and Programming (8086 to Pentium)*. Pearson Education. <https://books.google.ch/books?id=zWrZY1OgTPsC>
- [8] Leonardo De Moura and Nikolaj Björner. 2008. Z3: An efficient SMT solver. In *International conference on Tools and Algorithms for the Construction and Analysis of Systems*. Springer, 337–340.

- [9] Leonardo de Moura de Moura, Soonho Kong, Jeremy Avigad, Floris van Doorn, and Jakob von Raumer. [n. d.]. The Lean Theorem Prover (system description). Accessed: 2025-07-02.
- [10] Yuyou Fan and John Regehr. 2024. High-Throughput, Formal-Methods-Assisted Fuzzing for LLVM. In *Proceedings of the 2024 IEEE/ACM International Symposium on Code Generation and Optimization* (Edinburgh, United Kingdom) (CGO '24). IEEE Press, 349–358. doi:10.1109/CGO57630.2024.10444854
- [11] Justin M. Fargnoli and Alex E. MacLean. [n. d.]. A Beginner's Guide to SelectionDAG. <https://llvm.org/devmtg/2024-10/slides/tutorial/MacLean-Fargnoli-ABeginnersGuide-to-SelectionDAG.pdf>. Accessed: 2025-19-08.
- [12] Mathieu Fehr, Michel Weber, Christian Ulmann, Alexandre Lopoukhine, Martin Paul Lücke, Théo Degioanni, Christos Vasiladiotis, Michel Steuwer, and Tobias Grosser. 2025. xDSL: Sidekick Compilation for SSA-Based Compilers. In *Proceedings of the 23rd ACM/IEEE International Symposium on Code Generation and Optimization* (Las Vegas, NV, USA) (CGO '25). Association for Computing Machinery, New York, NY, USA, 179–192. doi:10.1145/3696443.3708945
- [13] Fehr, Matthieu. [n. d.]. Fuzzing tools for MLIR. <https://github.com/opencompl/mlir-fuzz>. Accessed: 2025-08-31.
- [14] Matt Fredrikson. 2022. Bug Catching: Automated Program Verification. <https://www.cs.cmu.edu/~15414/s23/lectures/21-bitblasting.pdf>. Accessed: 2025-08-22.
- [15] CNRS Inria and contributors. [n. d.]. The Rocq Prover. <https://rocq-prover.org/doc/V9.0.0/refman/index.html>. Accessed: 2025-08-25.
- [16] Fuqi Jia, Rui Han, Pei Huang, Minghao Liu, Feifei Ma, and Jian Zhang. 2023. Improving Bit-Blasting for Nonlinear Integer Constraints. In *Proceedings of the 32nd ACM SIGSOFT International Symposium on Software Testing and Analysis* (Seattle, WA, USA) (ISSTA 2023). Association for Computing Machinery, New York, NY, USA, 14–25. doi:10.1145/3597926.3598034
- [17] Chris Lattner and Vikram Adve. 2004. LLVM: A Compilation Framework for Lifelong Program Analysis & Transformation. In *Proceedings of the International Symposium on Code Generation and Optimization: Feedback-Directed and Runtime Optimization* (Palo Alto, California) (CGO '04). IEEE Computer Society, USA, 75.
- [18] Chris Lattner, Mehdi Amini, Uday Bondhugula, Albert Cohen, Andy Davis, Jacques Pienaar, River Riddle, Tatiana Shpeisman, Nicolas Vasilache, and Oleksandr Zinenko. 2021. MLIR: Scaling Compiler Infrastructure For Domain Specific Computation. In *2021 IEEE/ACM International Symposium on Code Generation and Optimization (CGO)*. 2–14. doi:10.48550/arXiv.2002.11054
- [19] Juneyoung Lee, Chung-Kil Hur, and Nuno P. Lopes. 2019. AliveInLean: A Verified LLVM Peephole Optimization Verifier. In *Proc. of the 31st International Conference on Computer-Aided Verification (CAV)*. doi:10.1007/978-3-030-25543-5_25
- [20] Xavier Leroy. [n. d.]. CompCert C: a trustworthy compiler. <https://compcert.org/man/manual001.html>. Accessed: 2025-07-07.
- [21] Xavier Leroy. 2009. Formal Verification Of A Realistic Compiler. In *Proceedings of the 35th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL)*. ACM, 1–12. doi:10.1145/1538788.1538814
- [22] LLVM Project. [n. d.]. LLVM Language Reference Manual. <https://llvm.org/docs/LangRef.html>. Accessed: 2025-05-28.
- [23] Nuno P Lopes, Juneyoung Lee, Chung-Kil Hur, Zhengyang Liu, and John Regehr. 2021. Alive2: Bounded Translation Validation For LLVM. In *Proceedings of the 42nd ACM SIGPLAN International Conference on Programming Language Design and Implementation*. ACM, 65–79. doi:10.1145/3453483.3454030
- [24] Nuno P. Lopes, David Menendez, Santosh Nagarakatte, and John Regehr. 2015. Provably correct peephole optimizations with alive. In *Proceedings of the 36th ACM SIGPLAN Conference on Programming Language Design and Implementation* (Portland, OR, USA) (PLDI '15). Association for Computing Machinery, New York, NY, USA, 22–32. doi:10.1145/2737924.2737965
- [25] Alexandre Lopoukhine, Federico Ficarelli, Christos Vasiladiotis, Anton Lydike, Josse Van Delm, Alban Dutilleul, Luca Benini, Marian Verhelst, and Tobias Grosser. 2025. A Multi-Level Compiler Backend for Accelerated Micro-kernels Targeting RISC-V ISA Extensions. In *CGO '25: Proceedings of the 23rd ACM/IEEE International Symposium on Code Generation and Optimization*. ACM, 163–178. doi:10.1145/3696443.3708952
- [26] Jia Meng, Claire Quigley, and Lawrence C. Paulson. 2006. Automation for interactive proof: First prototype. *Information and Computation* 204, 10 (2006), 1575–1596. doi:10.1016/j.ic.2005.05.010 Combining Logical Systems.
- [27] Eric Mullen, Daryl Zuniga, Zachary Tatlock, and Dan Grossman. 2016. Verified Peephole Optimizations For CompCert. In *PLDI '16: Proceedings of the 37th ACM SIGPLAN Conference on Programming Language Design and Implementation*. ACM, 448–461. doi:10.1145/1538788.1538814
- [28] Multi-Level IR Compiler Framework. [n. d.]. MLIR Language Reference Manual. <https://mlir.llvm.org/docs/DefiningDialects/AttributesAndTypes>. Accessed: 2025-07-07.
- [29] Multi-Level IR Compiler Framework. [n. d.]. Using mlir-opt. <https://mlir.llvm.org/docs/Tutorials/MlirOpt>. Accessed: 2025-08-17.
- [30] Julie L. Newcomb, Andrew Adams, Steven Johnson, Rastislav Bodik, and Shoaib Kamil. 2020. Verifying and improving Halide's term rewriting system with program synthesis. *Proc. ACM Program. Lang.* 4, OOPSLA, Article 166 (Nov. 2020), 28 pages. doi:10.1145/3428234

- [31] The Lean Focused Research Organization. [n. d.]. The Lean Language Reference. <https://lean-lang.org/doc/reference/latest/>. Accessed: 2025-08-17.
- [32] Louis-Emile Ploix, Alasdair Armstrong, Tom Melham, Ray Lin, Haolong Wang, and Anastasia Courtney. 2025. Comprehensive Formal Verification of Observational Correctness for the CHERIoT-Ibex Processor. In *Hardware Architecture*. 1–17. doi:10.48550/arXiv.2502.04738
- [33] John Regehr, Eugene Zelenko, Craig Topper, and Eli Friedman. [n. d.]. rem/div-related miscompile from riscv64 and aarch64 backends. <https://github.com/llvm/llvm-project/issues/145360>. Accessed: 2025-07-18.
- [34] Yuyang Rong, Zhanghan Yu, Zhenkai Weng, Stephen Neuendorffer, and Hao Chen. 2025. IRFuzzer: Specialized Fuzzing for LLVM Backend Code Generation . In *2025 IEEE/ACM 47th International Conference on Software Engineering (ICSE)*. IEEE Computer Society, Los Alamitos, CA, USA, 1986–1998. doi:10.1109/ICSE55347.2025.00130
- [35] D. Seal. 2001. *ARM Architecture Reference Manual*. Addison-Wesley. <https://books.google.ch/books?id=jhvIuwYdZ2UC>
- [36] Qian Wang. 2023. *Semantic Fuzzing Of The Rust Compiler And Interpreter*. Master’s thesis. ETH Zurich.
- [37] Robert N. M. Watson, Peter G. Neumann, Jonathan Woodruff, Michael Roe, Hesham Almatary, Jonathan Anderson, John Baldwin, David Chisnall, Brooks Davis, Nathaniel Wesley Filardo, Alexandre Joannou, Ben Laurie, A. Theodore Marketos, Simon W. Moore, Steven J. Murdoch, Kyndylan Nienhuis, Robert Norton, Alex Richardson, Peter Rugg, Peter Sewell, Stacey Son, and Hongyan Xia. 2019. *Capability Hardware Enhanced RISC Instructions: CHERI Instruction-Set Architecture (Version 7)*. Technical Report UCAM-CL-TR-927. University of Cambridge, Computer Laboratory. doi:10.48456/tr-927
- [38] Jianzhou Zhao, Santosh Nagarakatte, Milo M.K. Martin, and Steve Zdancewic. 2012. Formalizing the LLVM intermediate representation for verified program transformations. In *Proceedings of the 39th Annual ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages* (Philadelphia, PA, USA) (POPL ’12). Association for Computing Machinery, New York, NY, USA, 427–440. doi:10.1145/2103656.2103709